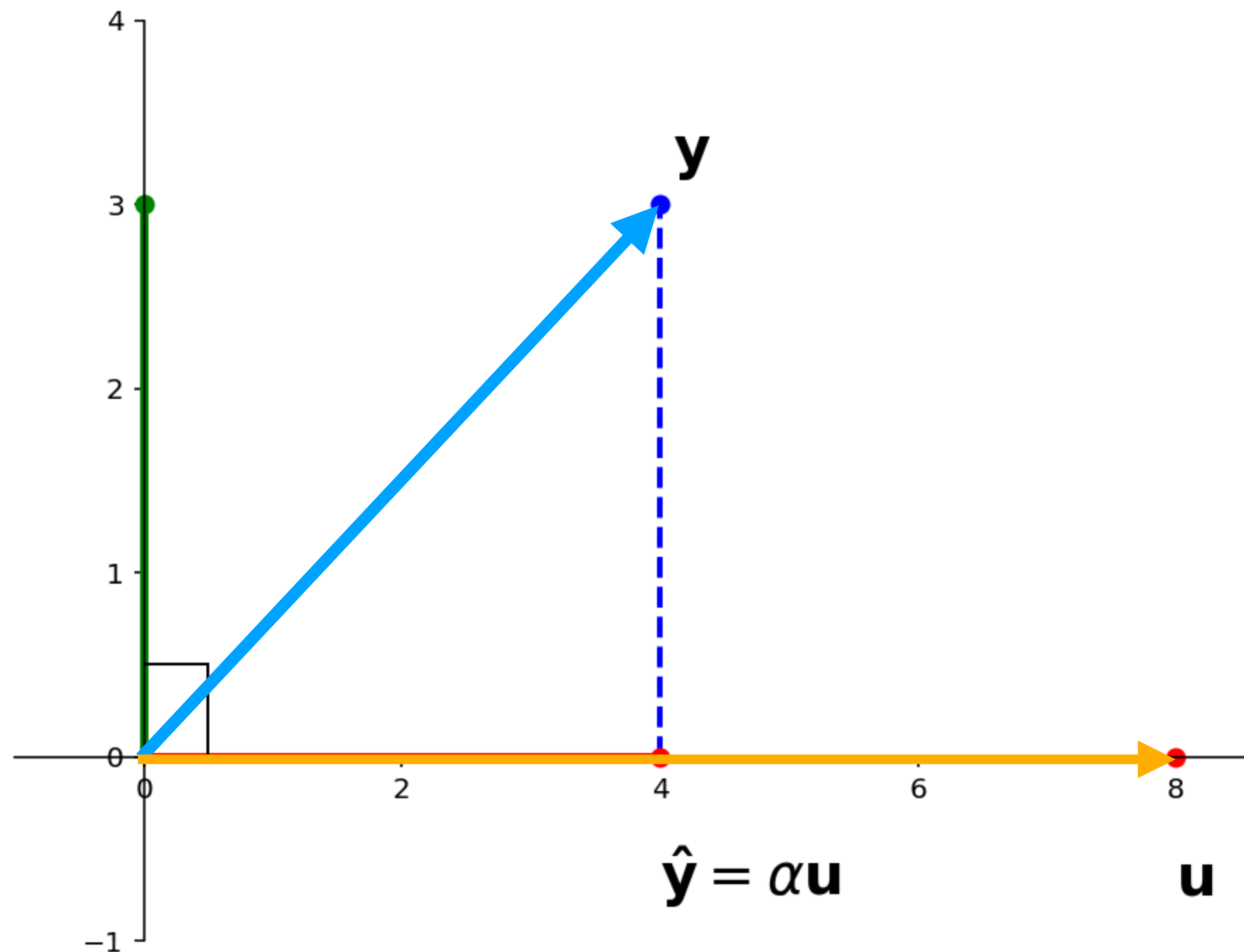


Least Squares

Geometric Algorithms

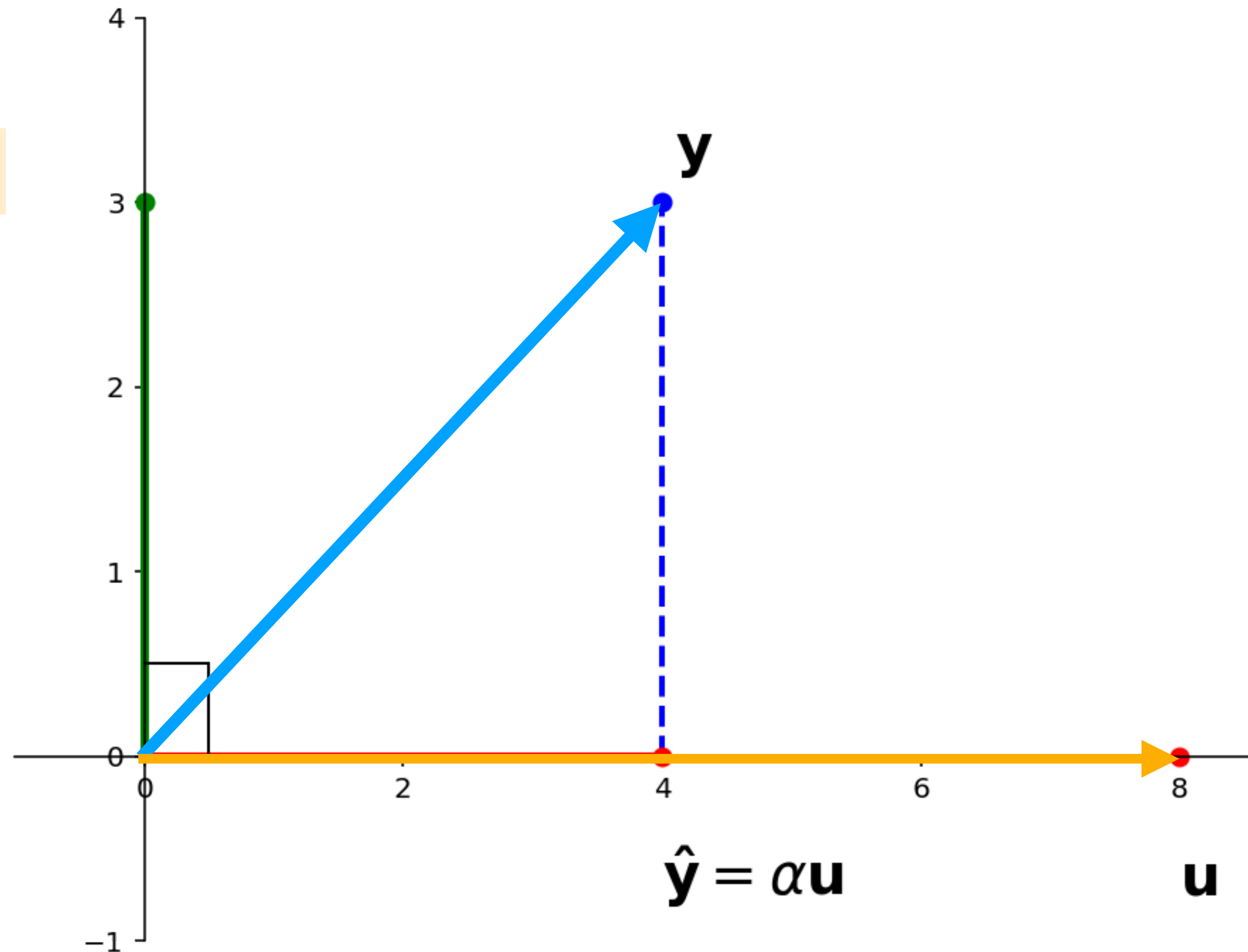
Recap

Recall: Orthogonal Projection



Recall: Orthogonal Projection

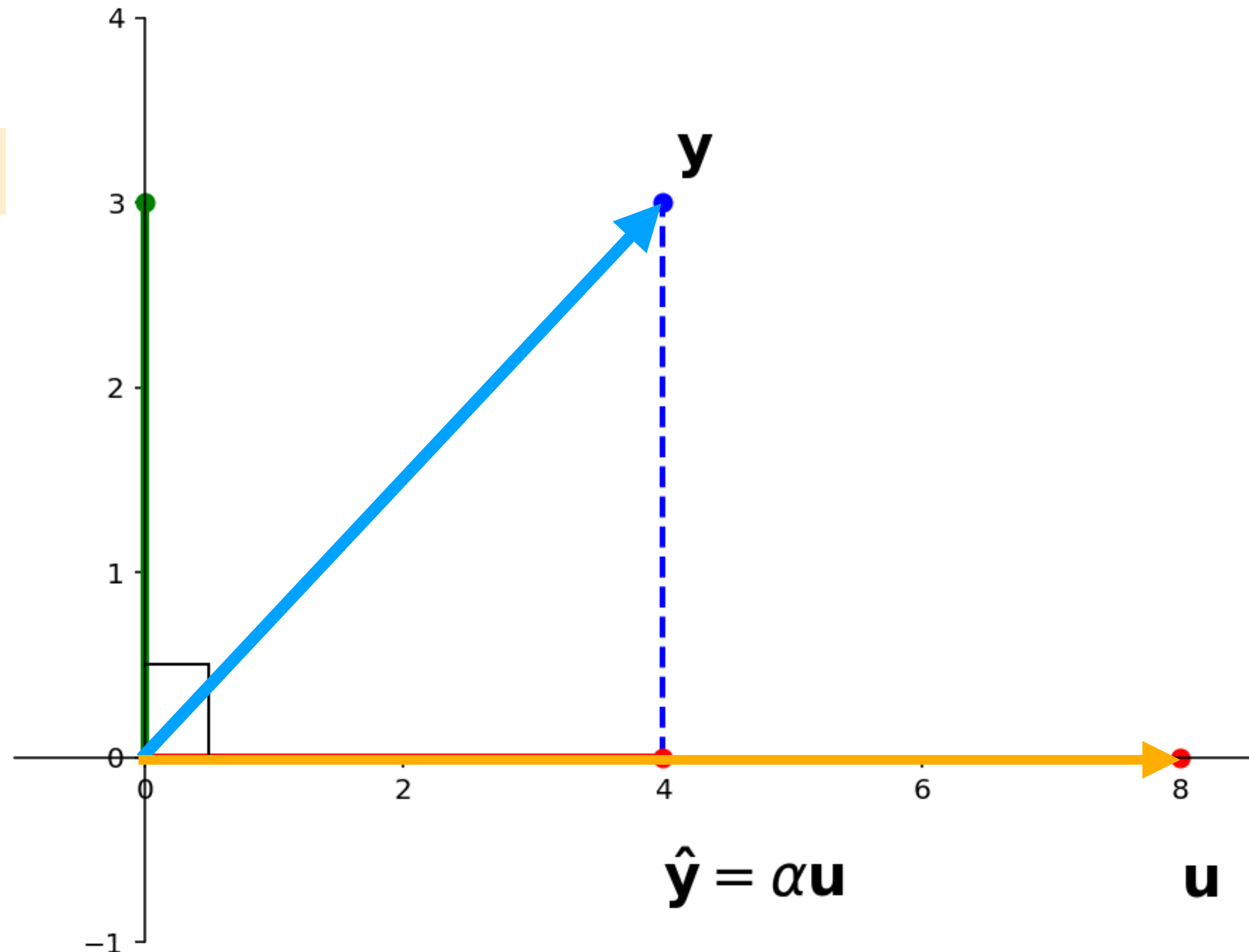
Given vectors y and u in $\mathbb{R}^n$, find vectors $\hat{y}$ and z such that:



Recall: Orthogonal Projection

Given vectors y and u in $\mathbb{R}^n$, find vectors $\hat{y}$ and z such that:

» z is orthogonal to u
(i.e., $z \cdot u = 0$)

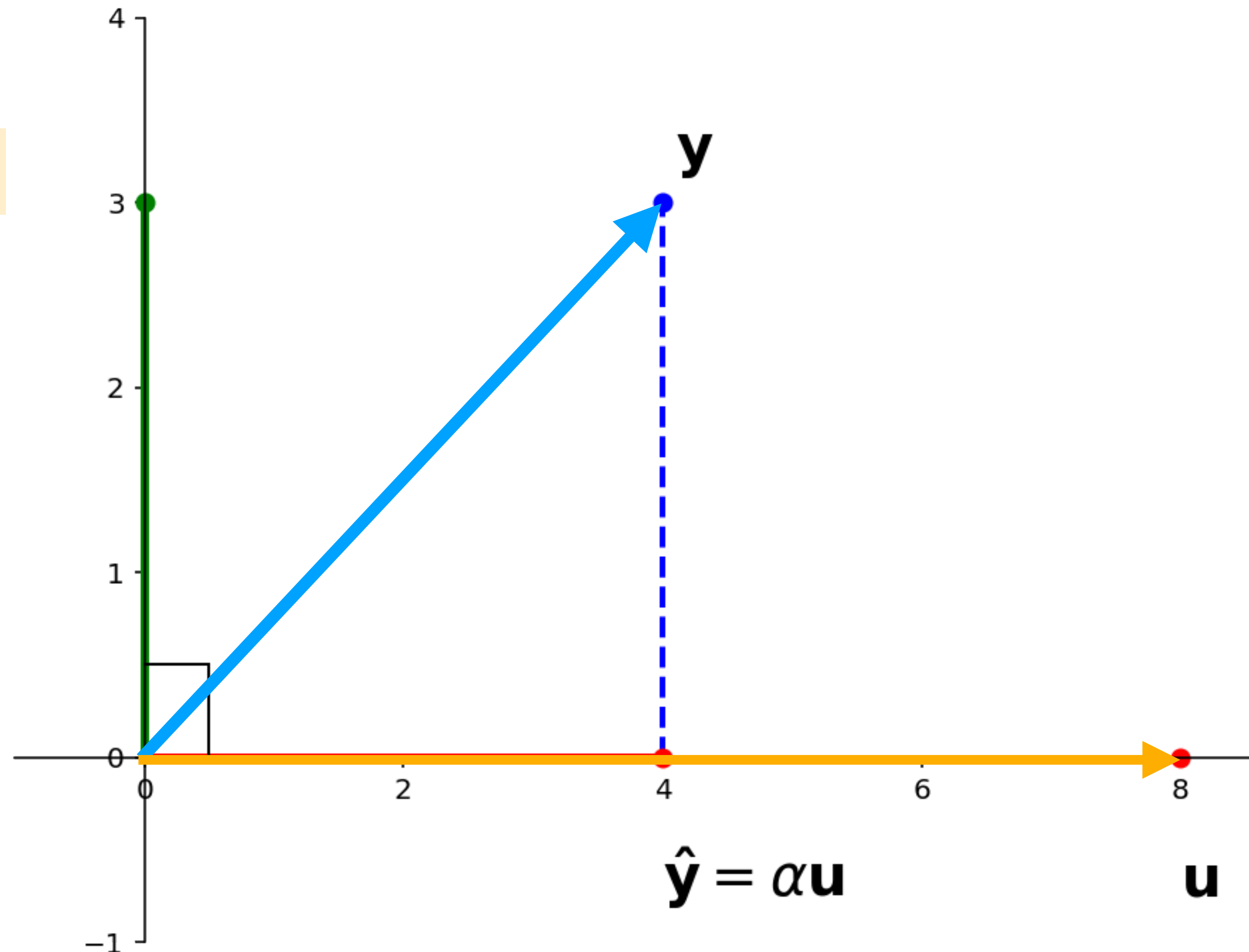


Recall: Orthogonal Projection

Given vectors y and u in $\mathbb{R}^n$, find vectors $\hat{y}$ and z such that:

» z is orthogonal to u
(i.e., $z \cdot u = 0$)

» $\hat{y} \in \text{span}\{u\}$



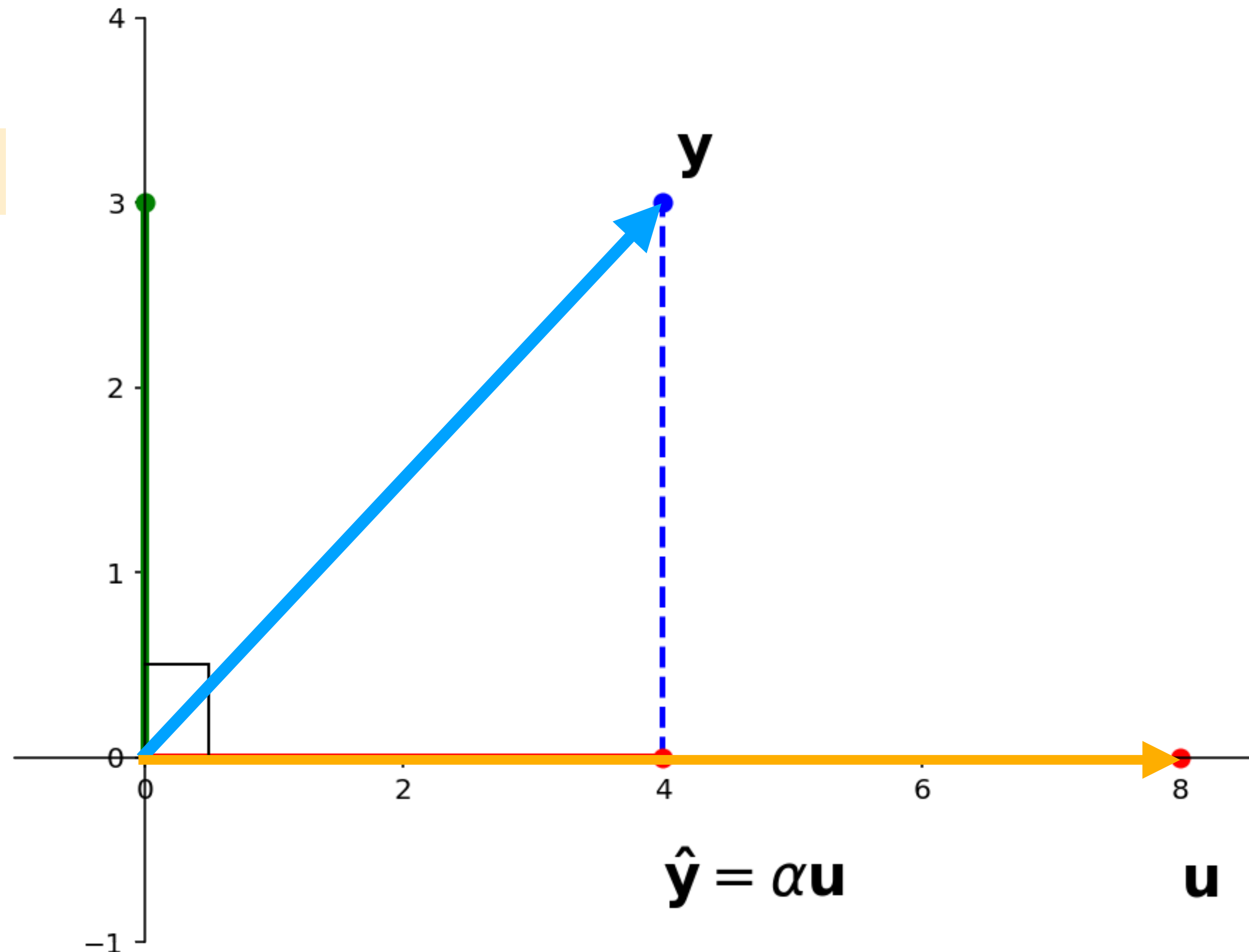
Recall: Orthogonal Projection

Given vectors y and u in $\mathbb{R}^n$, find vectors $\hat{y}$ and z such that:

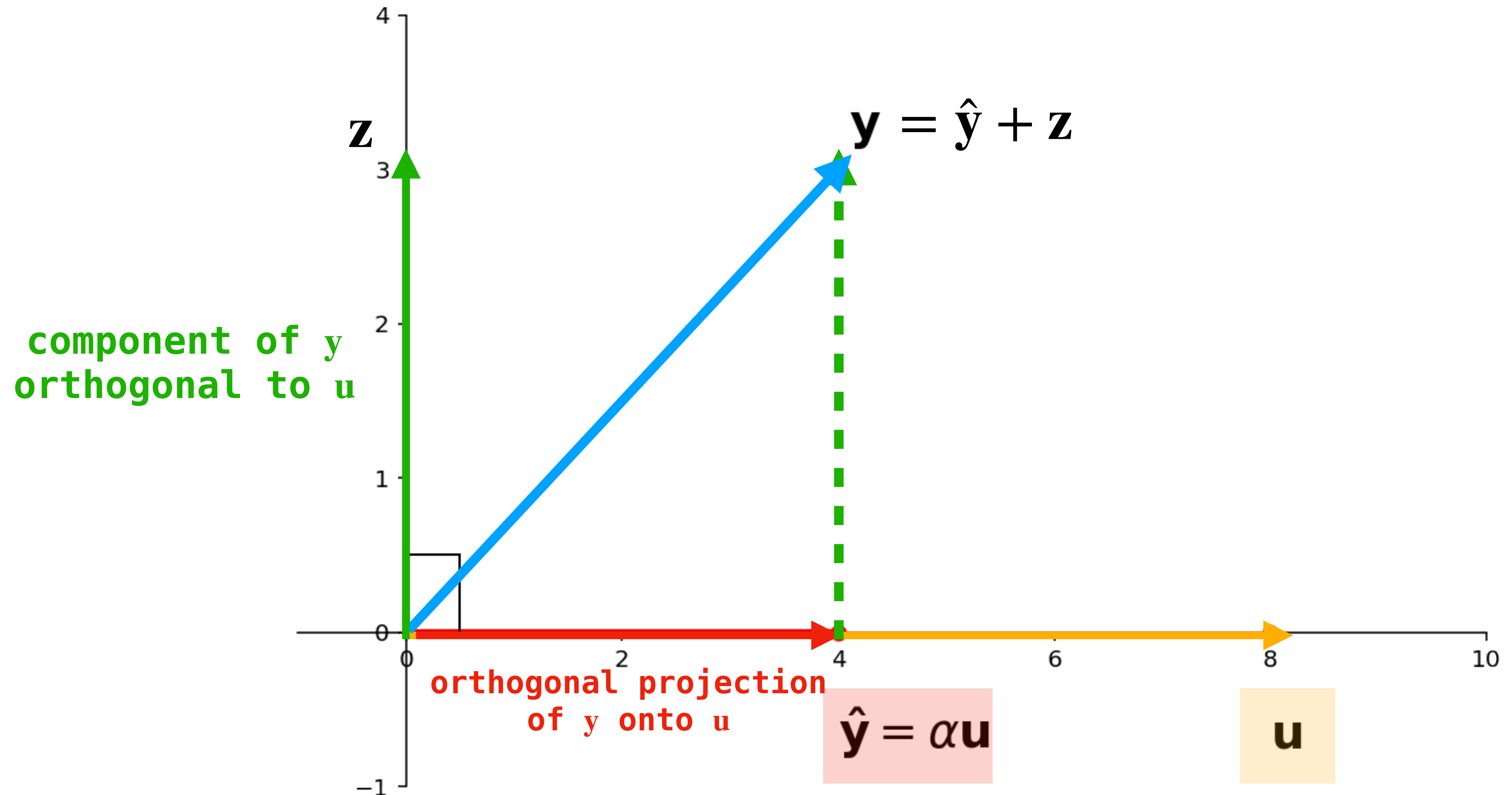
» z is orthogonal to u
(i.e., $z \cdot u = 0$)

» $\hat{y} \in \text{span}\{u\}$

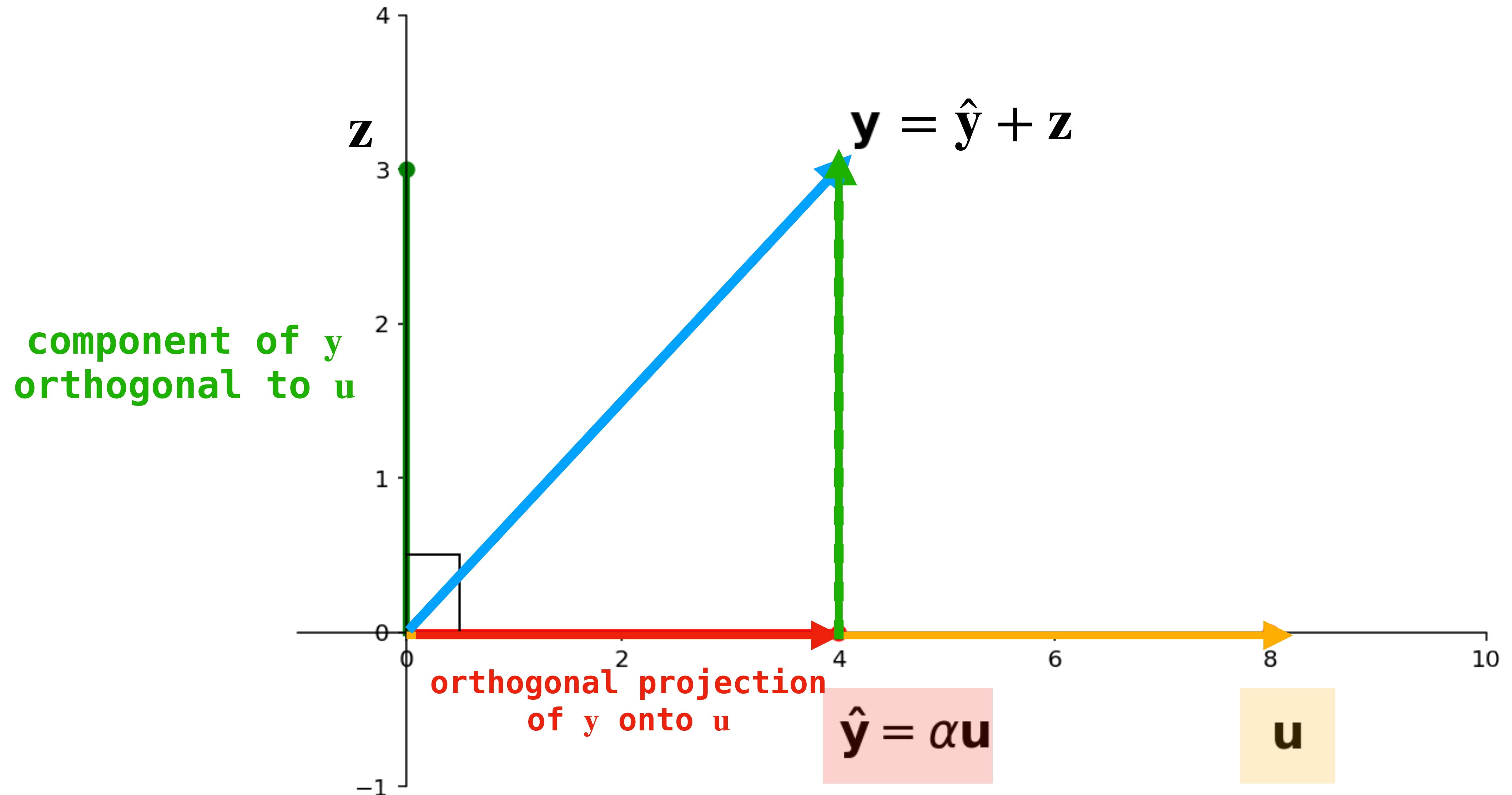
» $y = \hat{y} + z$



Orthogonal Projection

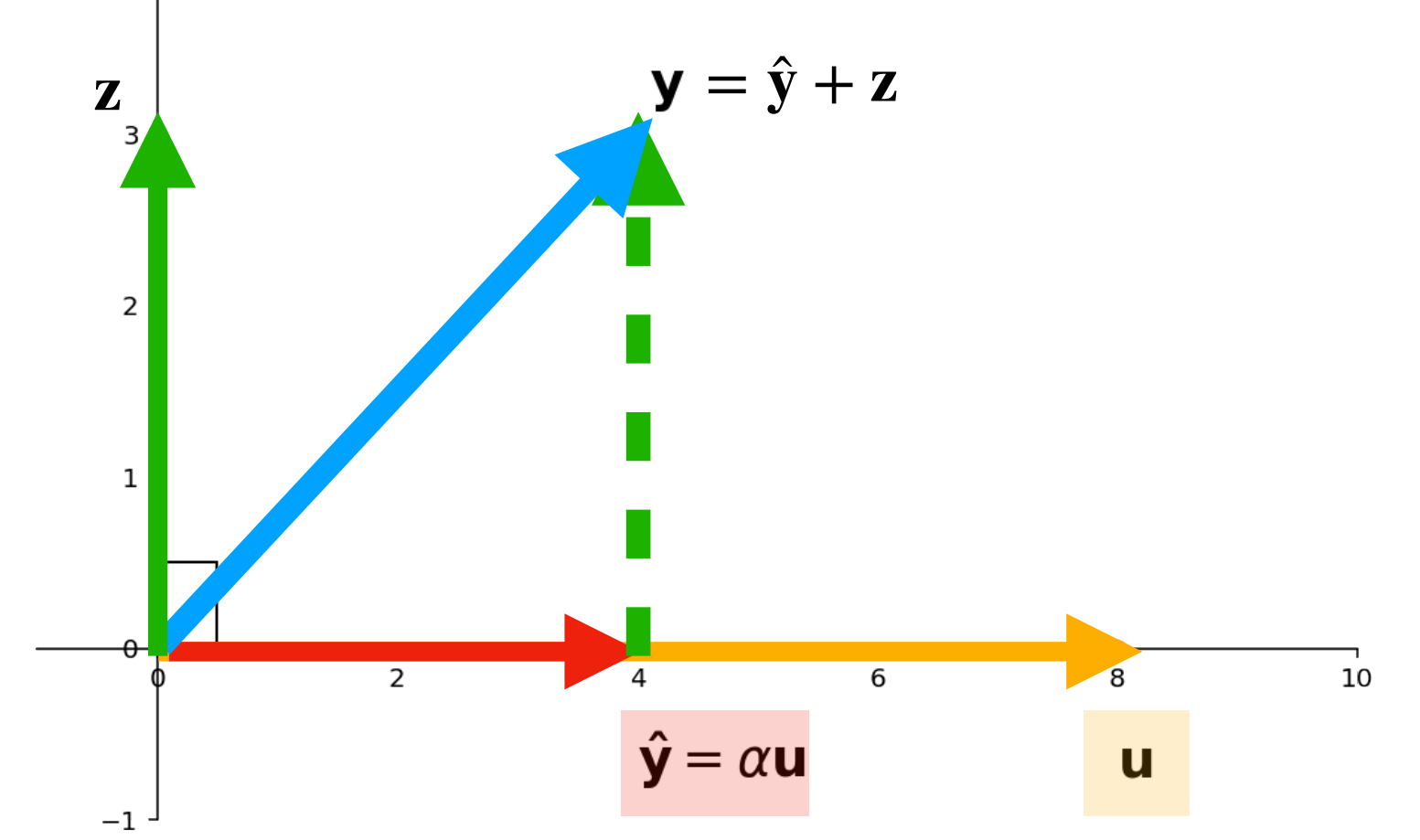


Orthogonal Projection



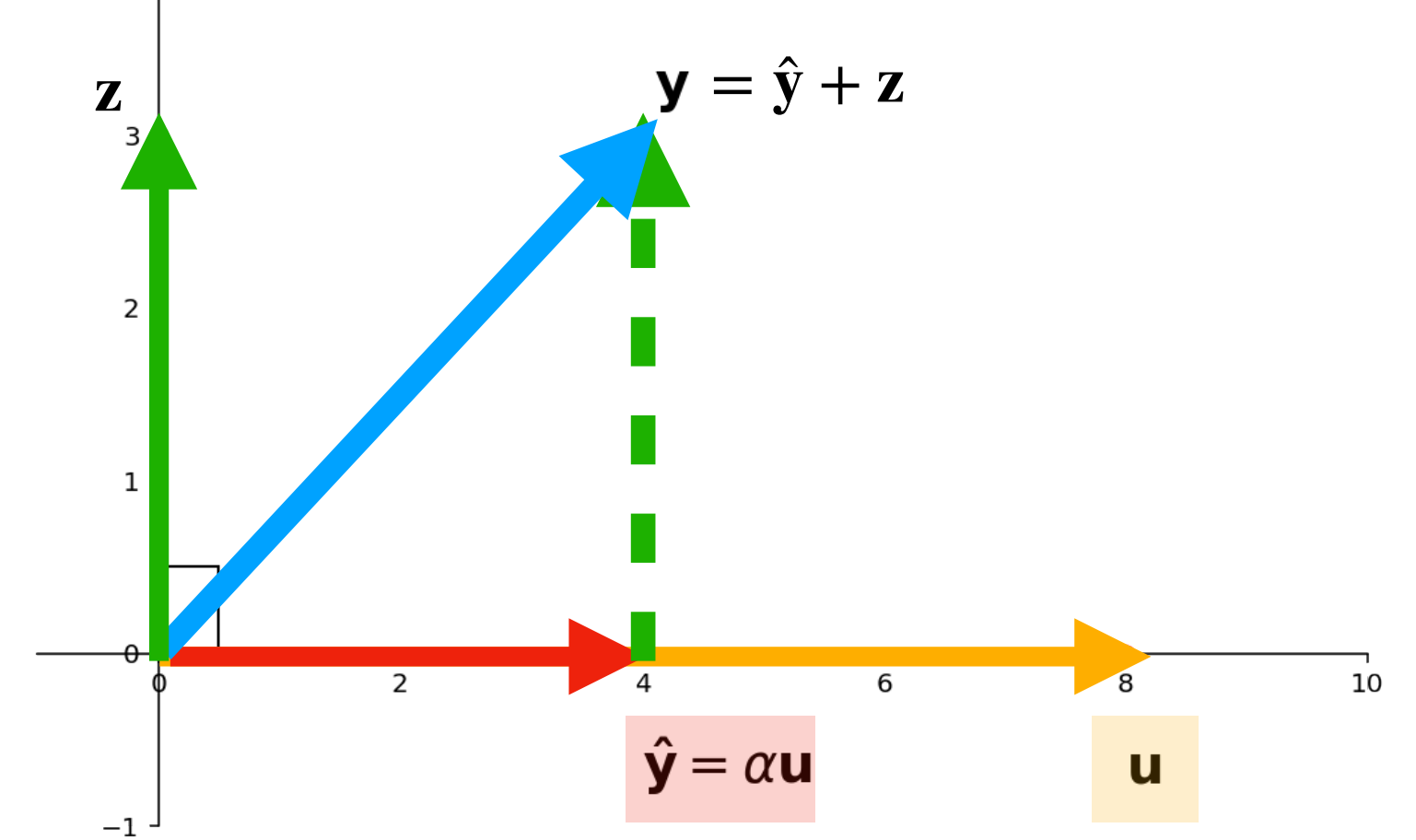
How do we find the orthogonal
projection and orthogonal component?

What we know

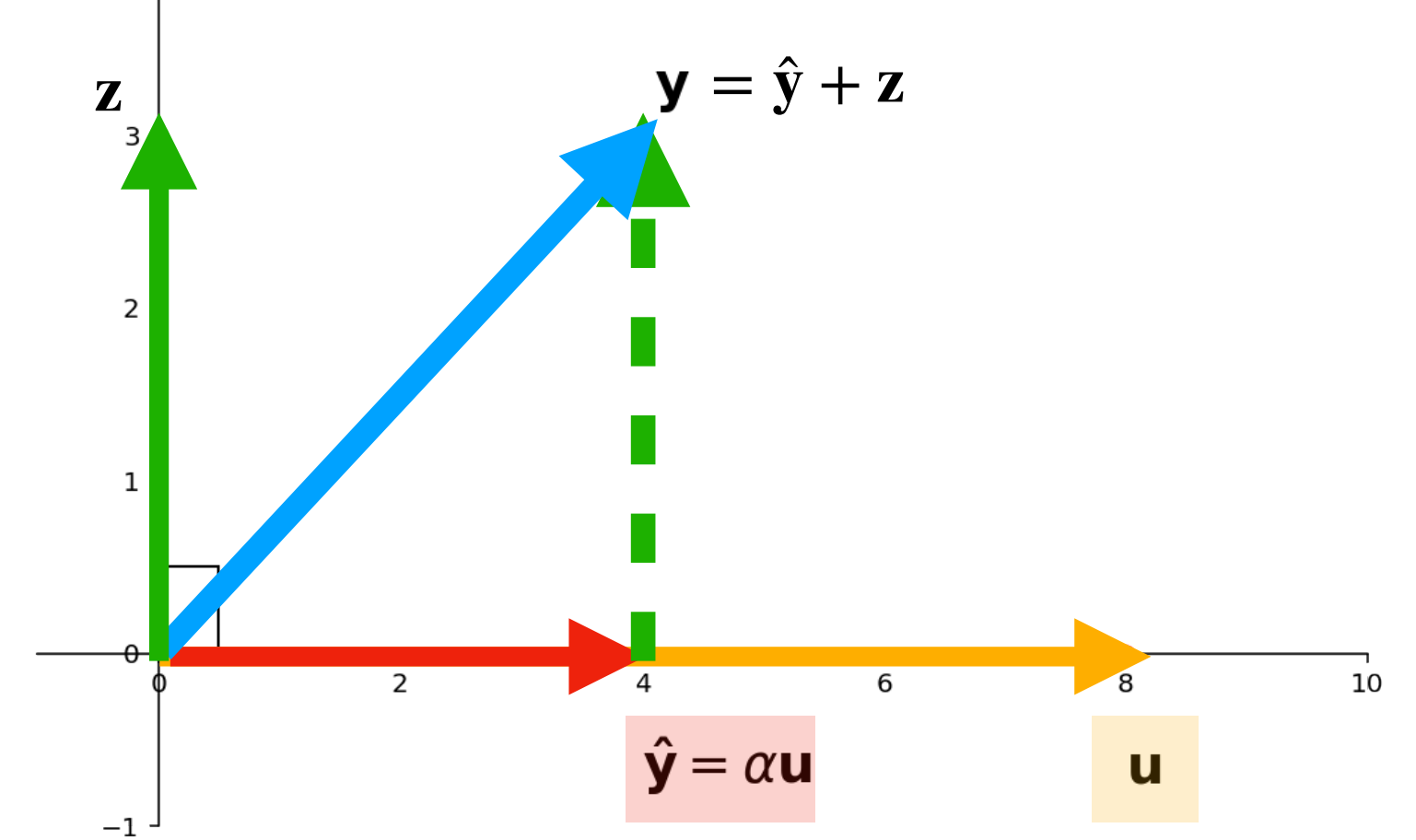


What we know

» $\hat{\mathbf{y}} = \alpha \mathbf{u}$ for some scalar α (since $\hat{\mathbf{y}} \in \text{span}\{\mathbf{u}\}$)



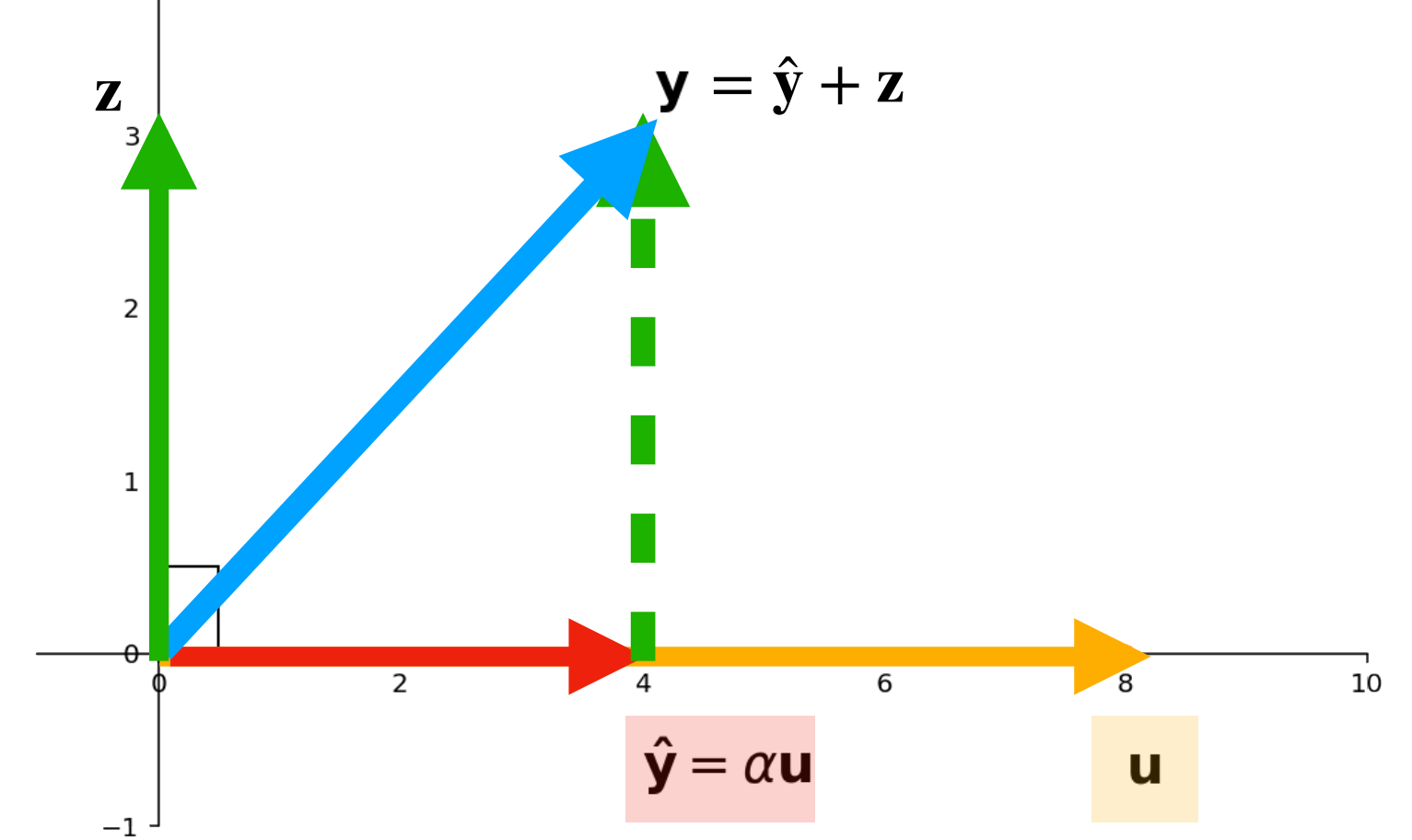
What we know



» $\hat{y} = \alpha u$ for some scalar α (since $\hat{y} \in \text{span}\{u\}$)

» $z = y - \hat{y} = y - \alpha u$ (since $y = \hat{y} + z$)

What we know

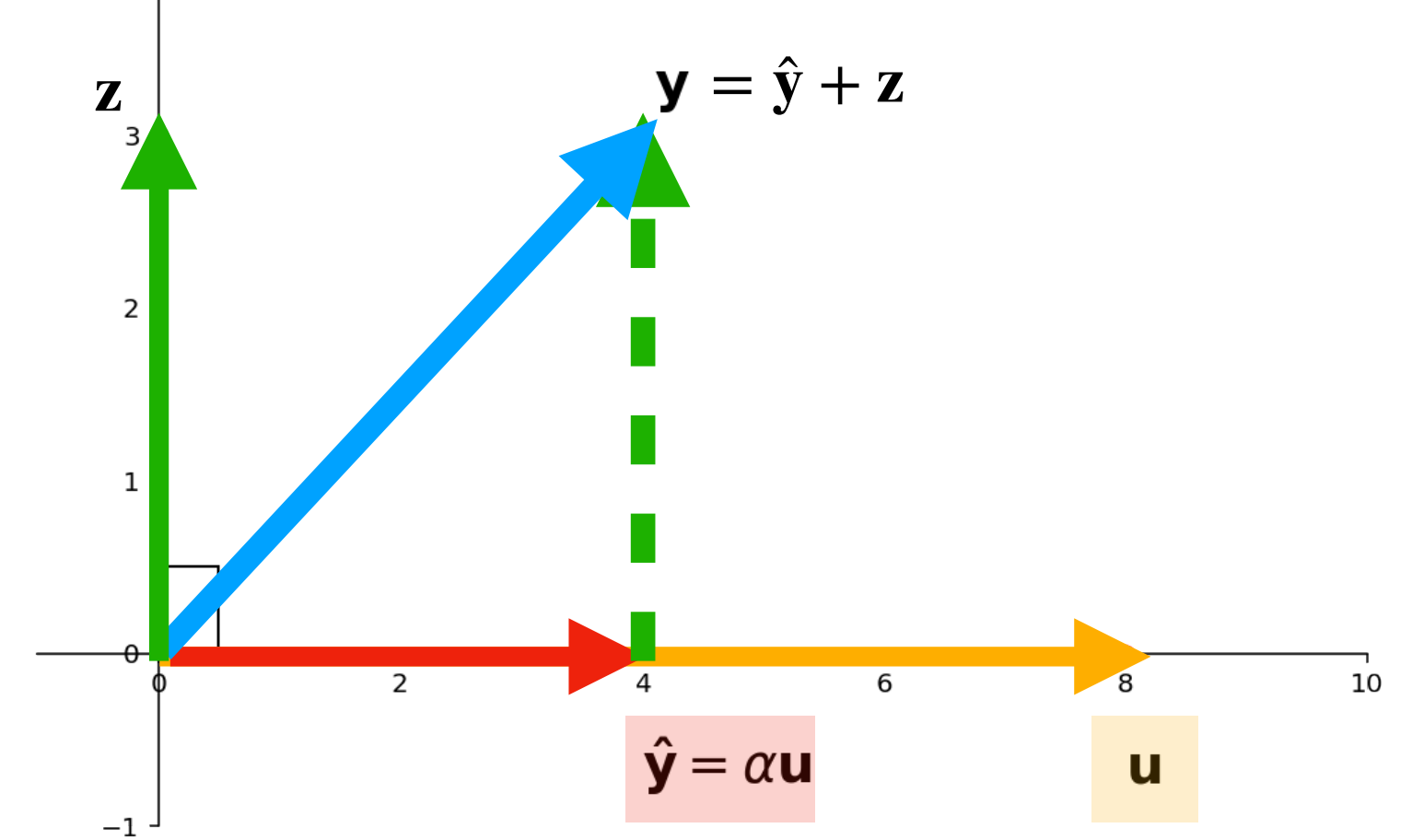


» $\hat{\mathbf{y}} = \alpha \mathbf{u}$ for some scalar α (since $\hat{\mathbf{y}} \in \text{span}\{\mathbf{u}\}$)

» $\mathbf{z} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \alpha \mathbf{u}$ (since $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{z}$)

» $\langle \mathbf{z}, \mathbf{u} \rangle = 0$ (since $\mathbf{z}$ is orthogonal with $\mathbf{u}$)

What we know



» $\hat{\mathbf{y}} = \alpha \mathbf{u}$ for some scalar α (since $\hat{\mathbf{y}} \in \text{span}\{\mathbf{u}\}$)

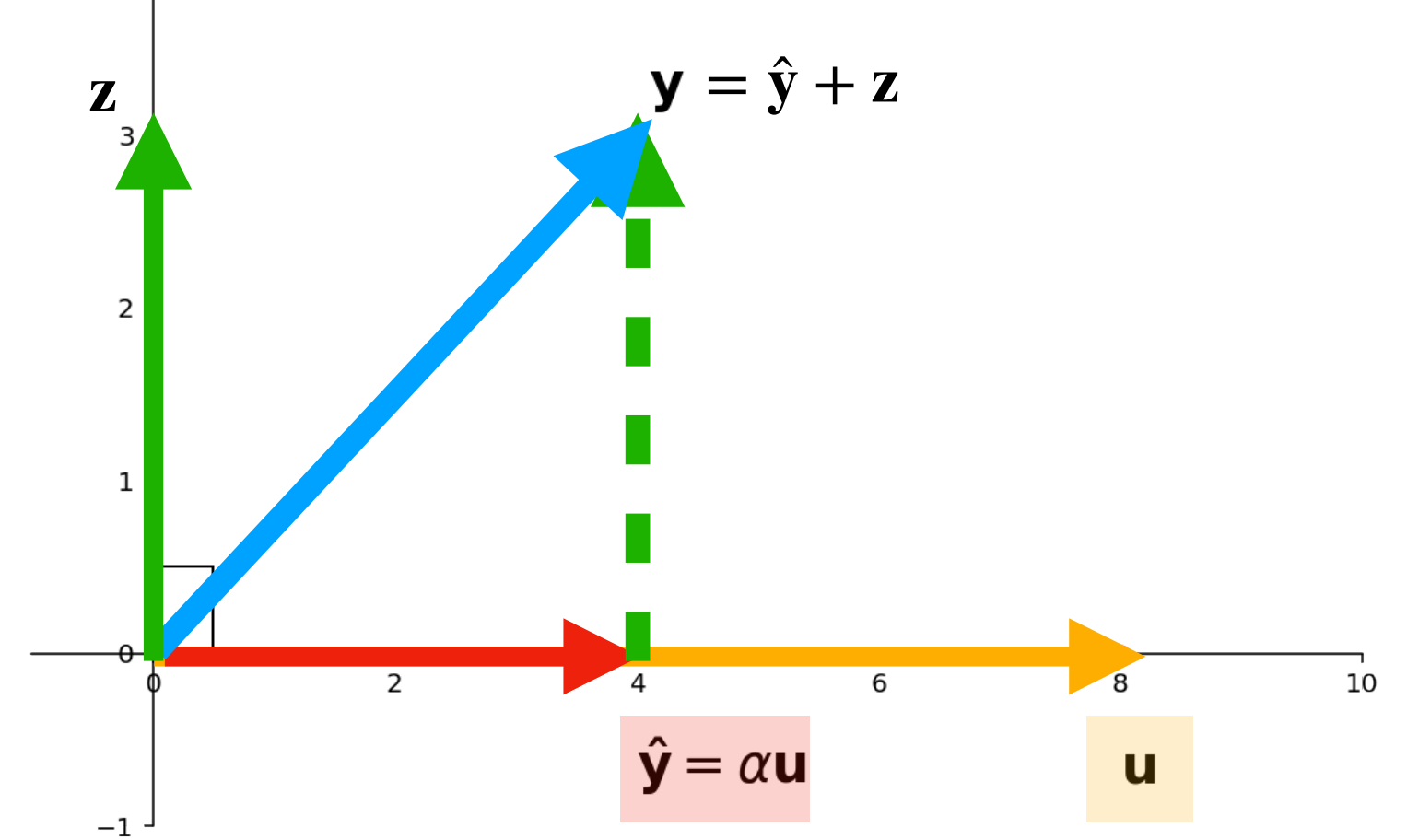
» $\mathbf{z} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \alpha \mathbf{u}$ (since $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{z}$)

» $\langle \mathbf{z}, \mathbf{u} \rangle = 0$ (since $\mathbf{z}$ is orthogonal with $\mathbf{u}$)

Therefore:

$$\langle \mathbf{y} - \alpha \mathbf{u}, \mathbf{u} \rangle = 0$$

What we know



- » $\hat{\mathbf{y}} = \alpha \mathbf{u}$ for some scalar α (since $\hat{\mathbf{y}} \in \text{span}\{\mathbf{u}\}$)
- » $\mathbf{z} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \alpha \mathbf{u}$ (since $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{z}$)
- » $\langle \mathbf{z}, \mathbf{u} \rangle = 0$ (since $\mathbf{z}$ is orthogonal with $\mathbf{u}$)

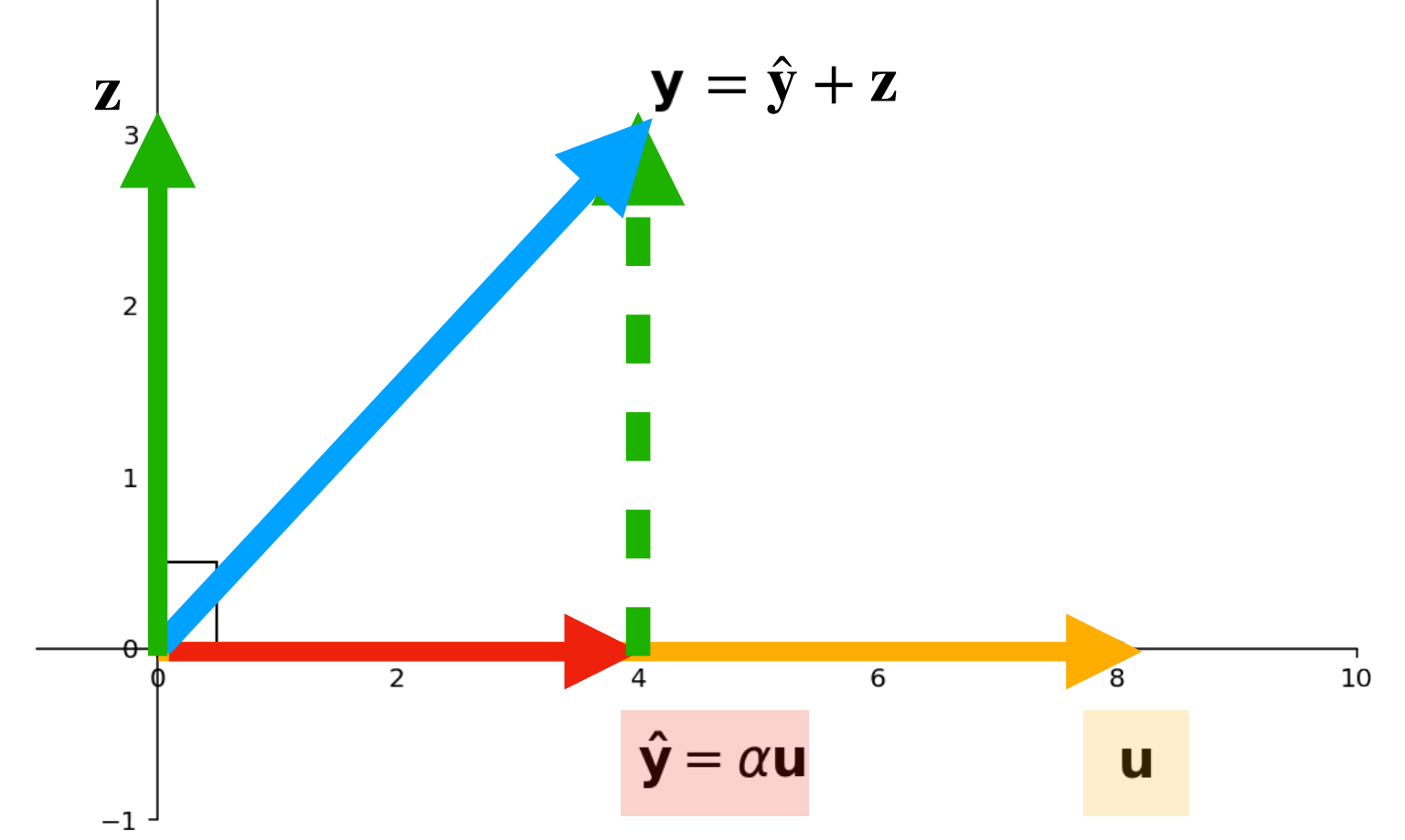
Therefore:

$$\langle \mathbf{y} - \alpha \mathbf{u}, \mathbf{u} \rangle = 0$$

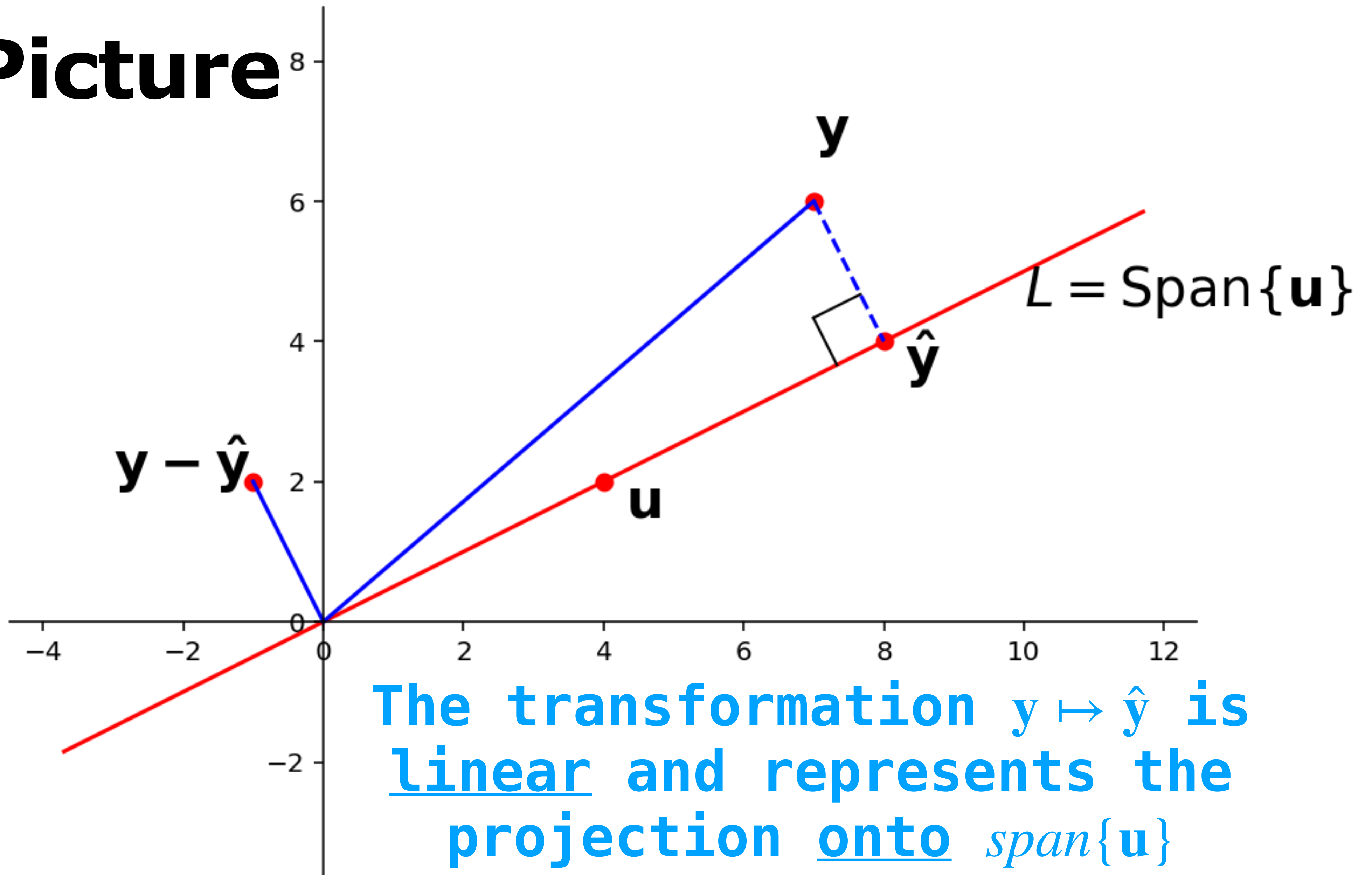
Once we have α , we can compute both $\hat{y}$ and z

Finding α

$$\langle \mathbf{y} - \alpha \mathbf{u}, \mathbf{u} \rangle = 0$$



The Picture



Recap Problem

$$\mathbf{u} = \begin{bmatrix} 1 \\ 3 \\ -2 \\ -1 \end{bmatrix} \quad \mathbf{v} = \begin{bmatrix} 0 \\ 1 \\ -1 \\ 0 \end{bmatrix}$$

Find the orthogonal projection of $\mathbf{u}$ onto the span of $\mathbf{v}$

Find the matrix which implements orthogonal projection onto the span of $\mathbf{v}$

Answer

\

$$\mathbf{u} = \begin{bmatrix} 1 \\ 3 \\ -2 \\ -1 \end{bmatrix} \quad \mathbf{v} = \begin{bmatrix} 0 \\ 1 \\ -1 \\ 0 \end{bmatrix}$$

Objectives

Introduce the **least squares problem** as a method of *approximating* solutions to matrix equations

Learn how to solve the least squares problems

Connect least squares solutions to projections

Keywords

general least squares problem

sum of squares error (ℓ_2 -error)

least squares solutions

orthogonal projections

normal equations

Motivation

The story of an enterprising CS132 student

The story of an enterprising CS132 student

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$

The story of an enterprising CS132 student

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$

A. Use `np.linalg.solve(A, b)`

The story of an enterprising CS132 student

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$

A. Use `np.linalg.solve(A, b)`

```
>>> A = np.array([
...     [1., 0, 5],
...     [1, -1, 4],
...     [0, 2, 2]])
>>> b = np.array([-1, 2, 3])
>>> np.linalg.solve(A, b)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/opt/homebrew/lib/python3.11/site-packages/numpy/linalg/linalg.py", line 409, in solve
    r = gufunc(a, b, signature=signature, extobj=extobj)
    ~~~~~^~~~~~
  File "/opt/homebrew/lib/python3.11/site-packages/numpy/linalg/linalg.py", line 112, in _raise_linalgerror_singular
    raise LinAlgError("Singular matrix")
numpy.linalg.LinAlgError: Singular matrix
```


The story of an enterprising CS132 student

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$

A. Use `np.linalg.solve(A, b)`

```
>>> A = np.array([
...     [1., 0, 5],
...     [1, -1, 4],
...     [0, 2, 2]])
>>> b = np.array([-1, 2, 3])
>>> np.linalg.solve(A, b)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/opt/homebrew/lib/python3.11/site-packages/numpy/linalg/linalg.py", line 409, in solve
    r = gufunc(a, b, signature=signature, extobj=extobj)
    ~~~~~^~~~~~
  File "/opt/homebrew/lib/python3.11/site-packages/numpy/linalg/linalg.py", line 112, in _raise_linalgerror_singular
    raise LinAlgError("Singular matrix")
numpy.linalg.LinAlgError: Singular matrix
```

This doesn't always work

Reads the docs...

numpy.linalg.solve

`linalg.solve(a, b)`

[\[source\]](#)

Solve a linear matrix equation, or system of linear scalar equations.

Computes the “exact” solution, x , of the well-determined, i.e., full rank, linear matrix equation $ax = b$.

Parameters: a : $(..., M, M)$ *array_like*

Coefficient matrix.

b : $\{(..., M,), (..., M, K)\}$, *array_like*

Ordinate or “dependent variable” values.

Returns: x : $\{(..., M,), (..., M, K)\}$ *ndarray*

Solution to the system $ax = b$. Returned shape is identical to b .

Raises: `LinAlgError`

If a is singular or not square.

 See also

[scipy.linalg.solve](#)

Similar function in SciPy

Reads the docs...

numpy.linalg.solve

`linalg.solve(a, b)`

[\[source\]](#)

Solve a linear matrix equation, or system of linear scalar equations.

Computes the “exact” solution, x , of the well-determined, i.e., full rank, linear matrix equation $ax = b$.

Parameters: a : $(..., M, M)$ *array_like*

Coefficient matrix.

b : $\{(..., M,), (..., M, K)\}$, *array_like*

Ordinate or “dependent variable” values.

Returns: x : $\{(..., M,), (..., M, K)\}$ *ndarray*

Solution to the system $ax = b$. Returned shape is identical to b .

Raises: `LinAlgError`

If a is singular or not square.

 See also

[scipy.linalg.solve](#)

Similar function in SciPy

Reads the docs...

Returns: `x : {(..., M,), (... , M, K)} ndarray`
Solution to the system $ax = b$. Returned shape is identical to b .

Raises: `LinAlgError`
If a is singular or not square.

See also

[`scipy.linalg.solve`](#)

Similar function in SciPy.

Notes

 *New in version 1.8.0.*

Broadcasting rules apply, see the [`numpy.linalg`](#) documentation for details.

The solutions are computed using LAPACK routine `_gesv`.

a must be square and of full-rank, i.e., all rows (or, equivalently, columns) must be linearly independent; if either is not true, use [`lstsq`](#) for the least-squares best “solution” of the system/equation.

Reads the docs...

Returns: `x : {(..., M,), (... , M, K)} ndarray`
Solution to the system $ax = b$. Returned shape is identical to b .

Raises: `LinAlgError`
If a is singular or not square.

See also

[`scipy.linalg.solve`](#)

Similar function in SciPy.

Notes

 *New in version 1.8.0.*

Broadcasting rules apply, see the [`numpy.linalg`](#) documentation for details.

The solutions are computed using LAPACK routine `_gesv`.

a must be square and of full-rank, i.e., all rows (or, equivalently, columns) must be linearly independent; if either is not true, use [`lstsq`](#) for the least-squares best “solution” of the system/equation.

np.linalg.lstsq

```
>>> np.linalg.lstsq(A, b)
<stdin>:1: FutureWarning: `rcond` parameter will change to the default of machine precision times ``max(M, N)``
where M and N are the input matrix dimensions.
To use the future default and silence this warning we advise to pass `rcond=None`, to keep using the old,
explicitly pass `rcond=-1`.
(array([-0.11111111,  0.77777778,  0.22222222]), array([], dtype=float64), 2, array([6.84168488e+00,
2.27845297e+00, 6.13801942e-17]))
>>> x = np.array([-0.11111111,  0.77777778,  0.22222222])
>>> A @ x
array([ 9.99999990e-01, -9.99999994e-09,  2.00000000e+00])
>>>
```

np.linalg.lstsq

```
>>> np.linalg.lstsq(A, b)
<stdin>:1: FutureWarning: `rcond` parameter will change to the default of machine precision times ``max(M, N)``
where M and N are the input matrix dimensions.
To use the future default and silence this warning we advise to pass `rcond=None`, to keep using the old,
explicitly pass `rcond=-1`.
(array([-0.11111111,  0.77777778,  0.22222222]), array([], dtype=float64), 2, array([6.84168488e+00,
2.27845297e+00, 6.13801942e-17]))
>>> x = np.array([-0.11111111,  0.77777778,  0.22222222])
>>> A @ x
array([ 9.99999990e-01, -9.99999994e-09,  2.00000000e+00])
>>>
```

np.linalg.lstsq

```
>>> np.linalg.lstsq(A, b)
<stdin>:1: FutureWarning: `rcond` parameter will change to the default of machine precision times ``max(M, N)``
where M and N are the input matrix dimensions.
To use the future default and silence this warning we advise to pass `rcond=None`, to keep using the old,
explicitly pass `rcond=-1`.
(array([-0.11111111,  0.77777778,  0.22222222]), array([], dtype=float64), 2, array([6.84168488e+00,
2.27845297e+00, 6.13801942e-17]))
>>> x = np.array([-0.11111111,  0.77777778,  0.22222222])
>>> A @ x
array([ 9.99999990e-01, -9.99999994e-09,  2.00000000e+00])
>>>
```


np.linalg.lstsq

```
>>> np.linalg.lstsq(A, b)
<stdin>:1: FutureWarning: `rcond` parameter will change to the default of machine precision times ``max(M, N)``
where M and N are the input matrix dimensions.
To use the future default and silence this warning we advise to pass `rcond=None`, to keep using the old,
explicitly pass `rcond=-1`.
(array([-0.11111111,  0.77777778,  0.22222222]), array([], dtype=float64), 2, array([6.84168488e+00,
2.27845297e+00, 6.13801942e-17]))
>>> x = np.array([-0.11111111,  0.77777778,  0.22222222])
>>> A @ x
array([ 9.99999990e-01, -9.99999994e-09,  2.00000000e+00])
>>>
```

np.linalg.lstsq

```
>>> np.linalg.lstsq(A, b)
<stdin>:1: FutureWarning: `rcond` parameter will change to the default of machine precision times ``max(M, N)``
where M and N are the input matrix dimensions.
To use the future default and silence this warning we advise to pass `rcond=None`, to keep using the old,
explicitly pass `rcond=-1`.
(array([-0.11111111,  0.77777778,  0.22222222]), array([], dtype=float64), 2, array([6.84168488e+00,
2.27845297e+00, 6.13801942e-17]))
>>> x = np.array([-0.11111111,  0.77777778,  0.22222222])
>>> A @ x
array([ 9.99999990e-01, -9.99999994e-09,  2.00000000e+00])
>>>
```

np.linalg.lstsq

```
>>> np.linalg.lstsq(A, b)
<stdin>:1: FutureWarning: `rcond` parameter will change to the default of machine precision times ``max(M, N)``
where M and N are the input matrix dimensions.
To use the future default and silence this warning we advise to pass `rcond=None`, to keep using the old,
explicitly pass `rcond=-1`.
(array([-0.11111111,  0.77777778,  0.22222222]), array([], dtype=float64), 2, array([6.84168488e+00,
2.27845297e+00, 6.13801942e-17]))
>>> x = np.array([-0.11111111,  0.77777778,  0.22222222])
>>> A @ x
array([ 9.99999990e-01, -9.99999994e-09,  2.00000000e+00])
>>>
```

uh...probably numerical errors...

Final answer: $\mathbf{x} = \begin{bmatrix} -1/9 \\ 7/9 \\ 2/9 \end{bmatrix}$

np.linalg.lstsq

```
>>> np.linalg.lstsq(A, b)
<stdin>:1: FutureWarning: `rcond` parameter will change to the default of machine precision times ``max(M, N)``
where M and N are the input matrix dimensions.
To use the future default and silence this warning we advise to pass `rcond=None`, to keep using the old,
explicitly pass `rcond=-1`.
(array([-0.11111111,  0.77777778,  0.22222222]), array([], dtype=float64), 2, array([6.84168488e+00,
2.27845297e+00, 6.13801942e-17]))
>>> x = np.array([-0.11111111,  0.77777778,  0.22222222])
>>> A @ x
array([ 9.99999990e-01, -9.99999994e-09,  2.00000000e+00])
>>>
```

uh...probably numerical errors...

Final answer: $\mathbf{x} = \begin{bmatrix} -1/9 \\ 7/9 \\ 2/9 \end{bmatrix}$ **This is not correct**

This System is Inconsistent

$$\begin{bmatrix} 1 & 0 & 5 & -1 \\ 1 & -1 & 4 & 2 \\ 0 & 2 & 2 & 3 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & 5 & -1 \\ 0 & -1 & -1 & 3 \\ 0 & 2 & 2 & 3 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & 5 & -1 \\ 0 & -1 & -1 & 3 \\ 0 & 0 & 0 & 9 \end{bmatrix}$$

The correct answer: There is no solution

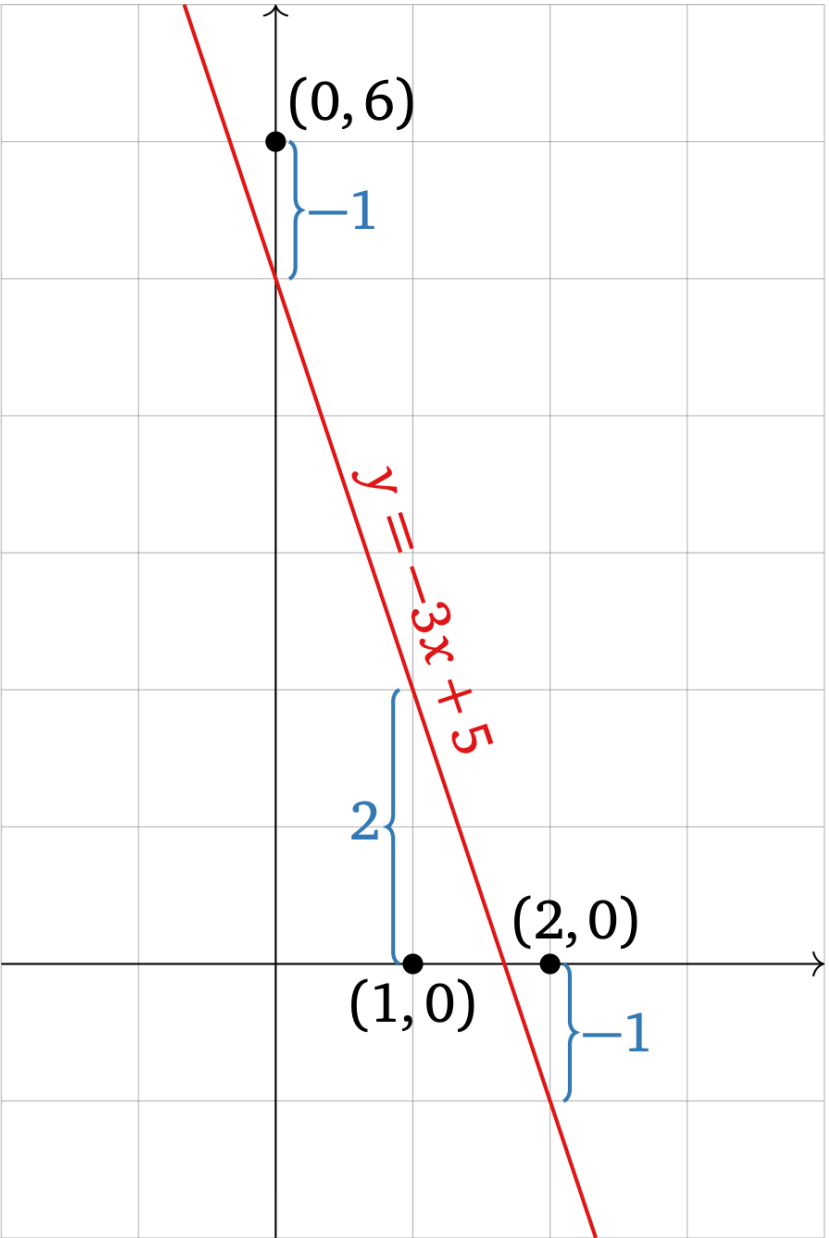
This System is Inconsistent

$$\begin{bmatrix} 1 & 0 & 5 & -1 \\ 1 & -1 & 4 & 2 \\ 0 & 2 & 2 & 3 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & 5 & -1 \\ 0 & -1 & -1 & 3 \\ 0 & 2 & 2 & 3 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & 5 & -1 \\ 0 & -1 & -1 & 3 \\ 0 & 0 & 0 & 9 \end{bmatrix}$$

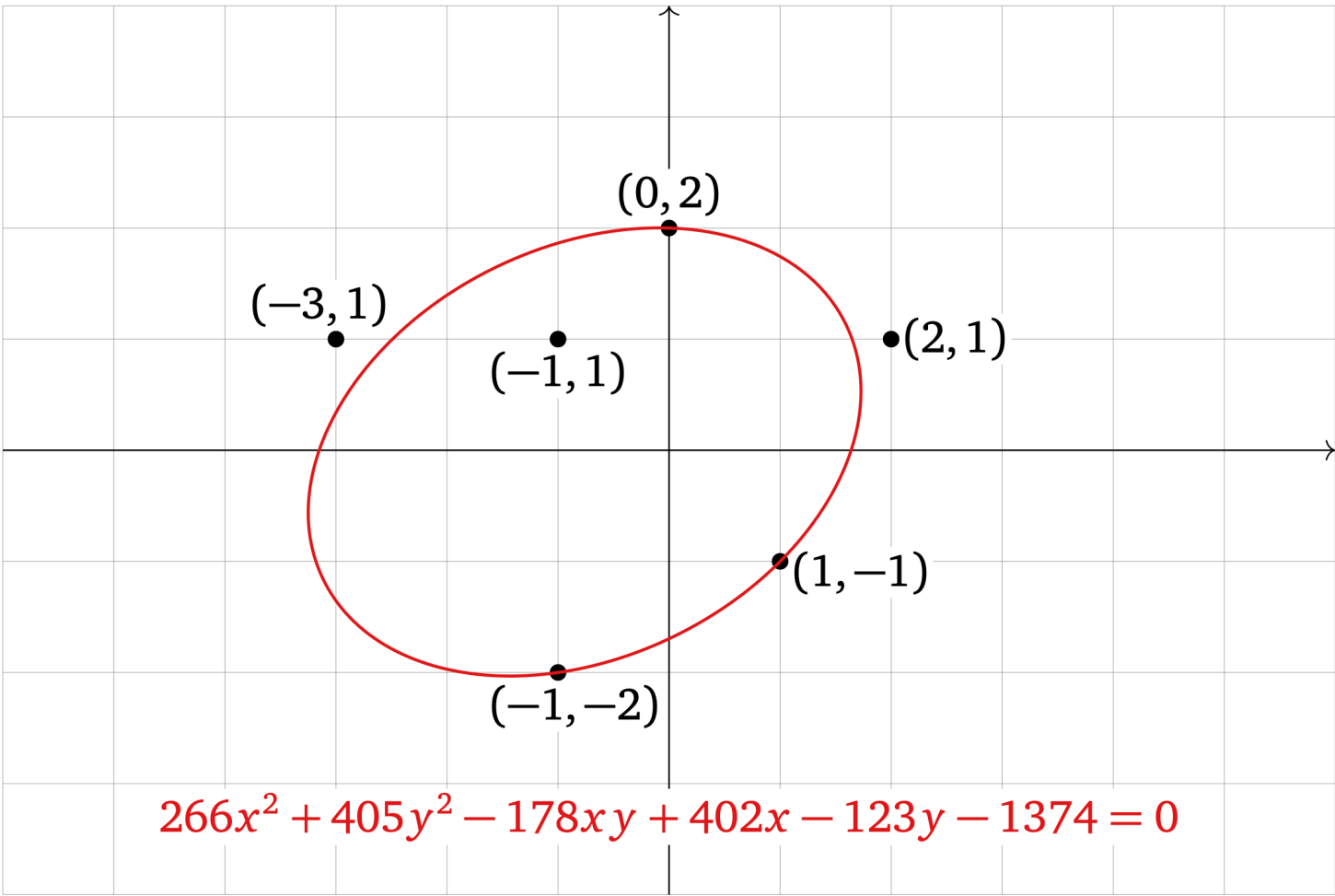
The correct answer: There is no solution

What's going on here?

Non-Linearity

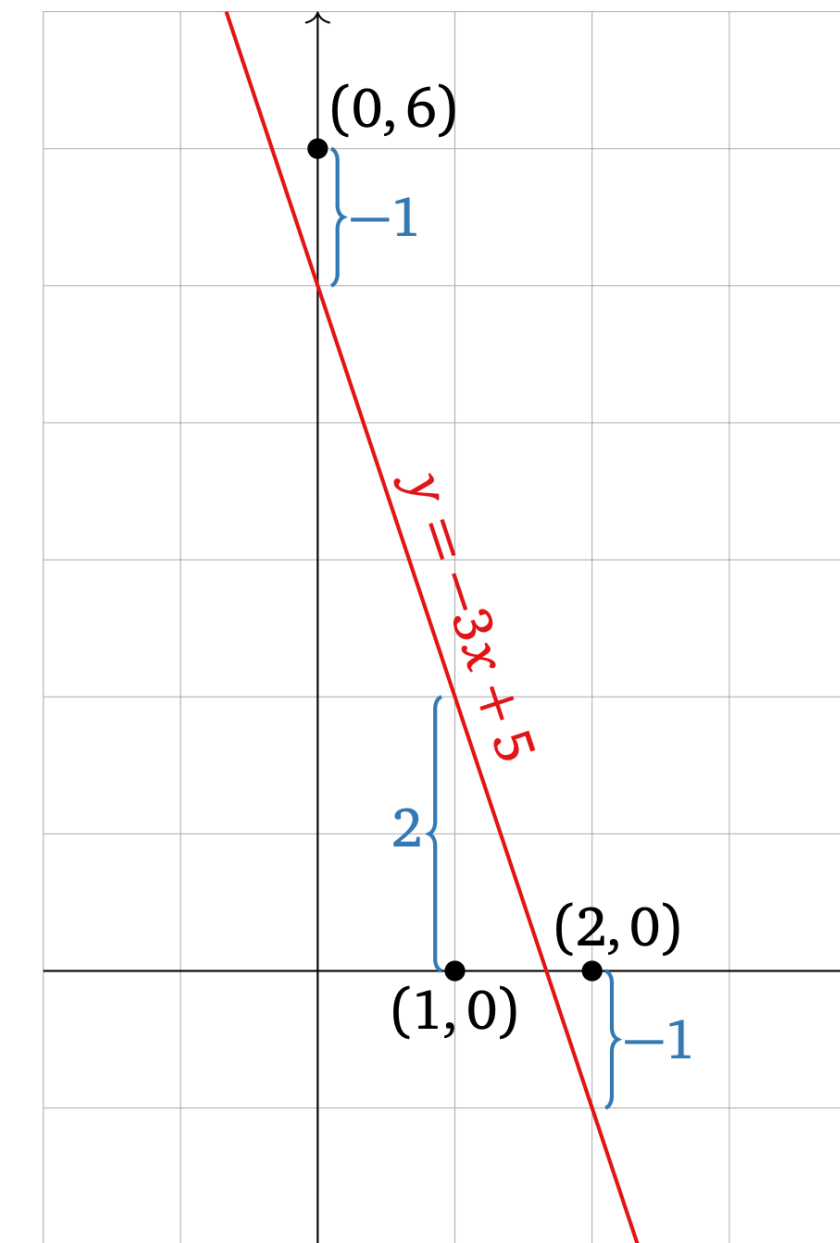


$$b - A\hat{x} = \begin{pmatrix} 6 \\ 0 \\ 0 \end{pmatrix} - A \begin{pmatrix} -3 \\ 5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$$

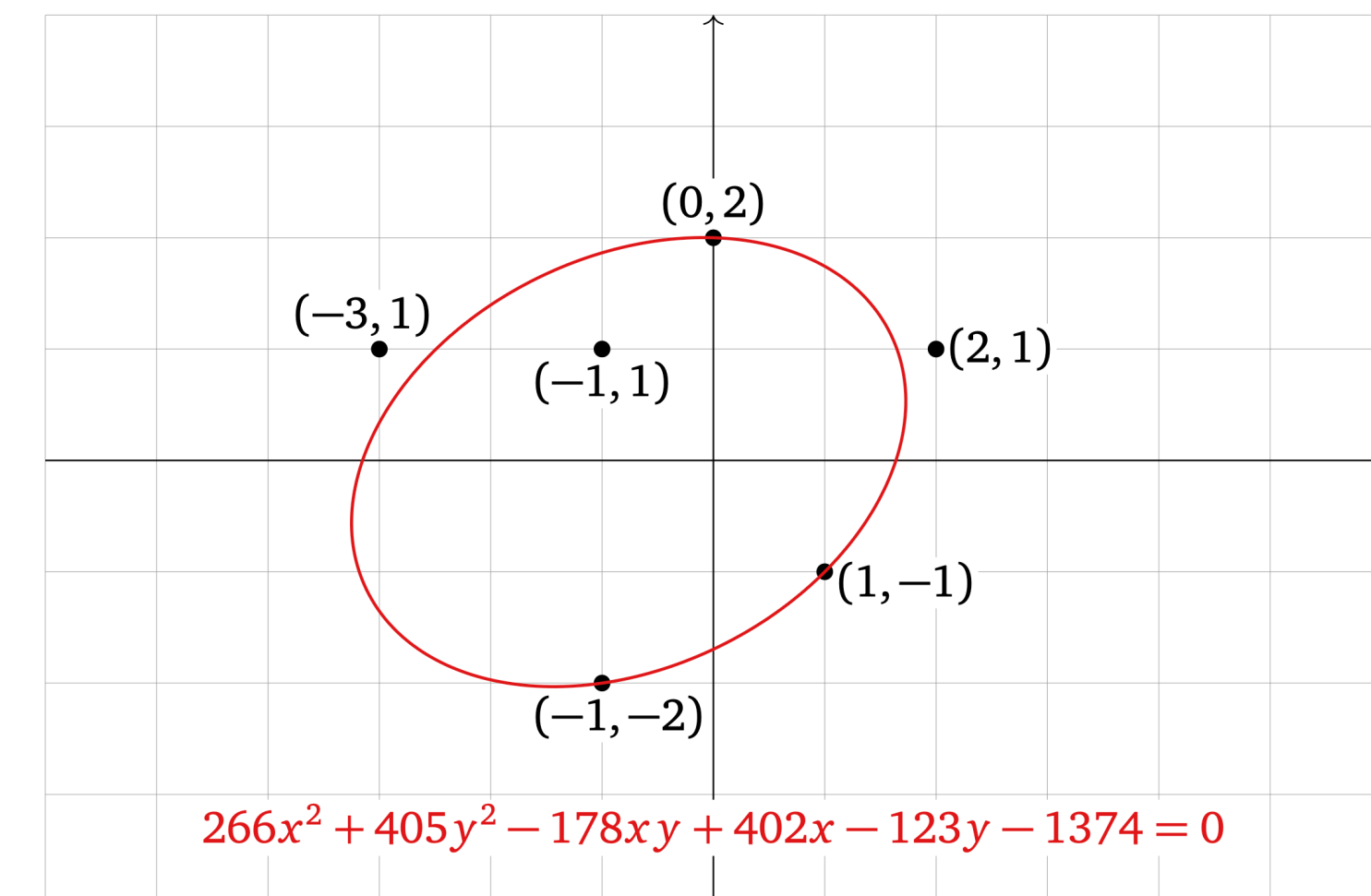


Non-Linearity

Linear algebra is powerful, but
the world isn't linear



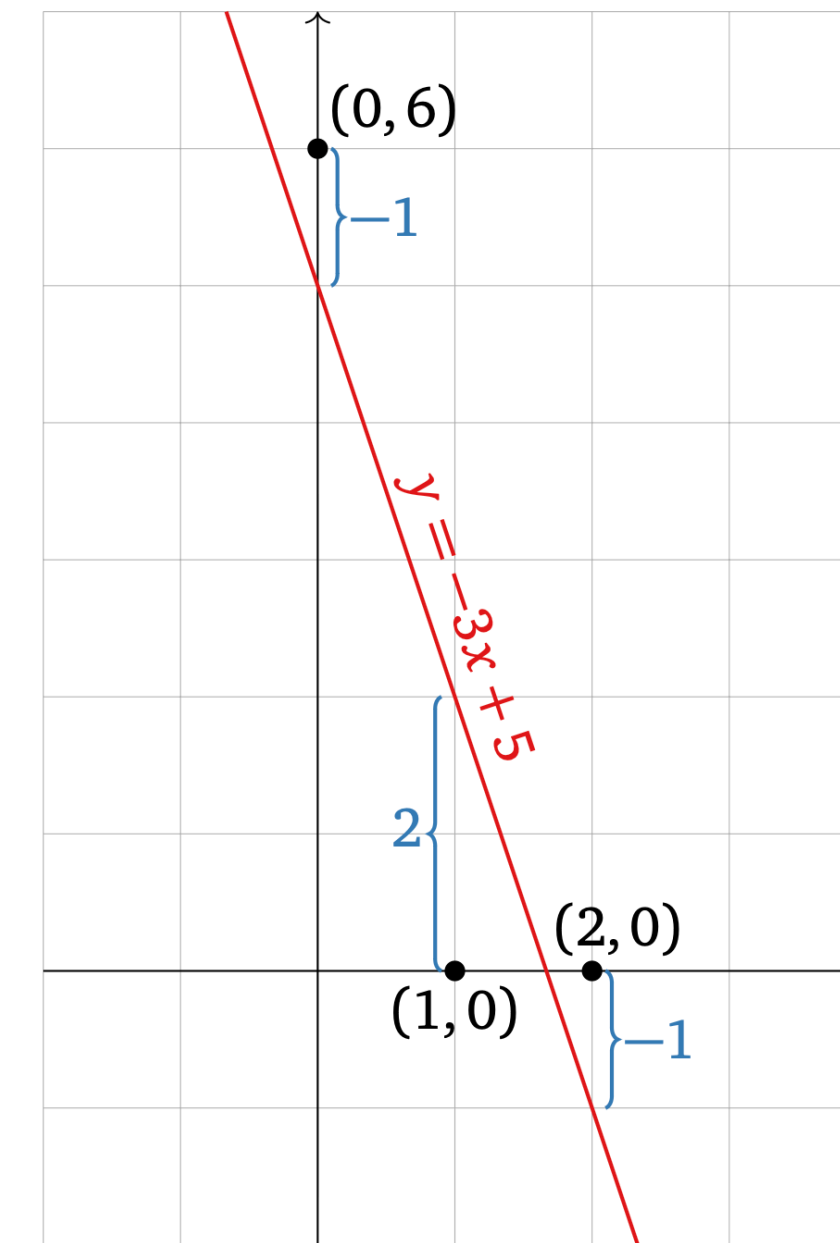
$$b - A\hat{x} = \begin{pmatrix} 6 \\ 0 \\ 0 \end{pmatrix} - A \begin{pmatrix} -3 \\ 5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$$



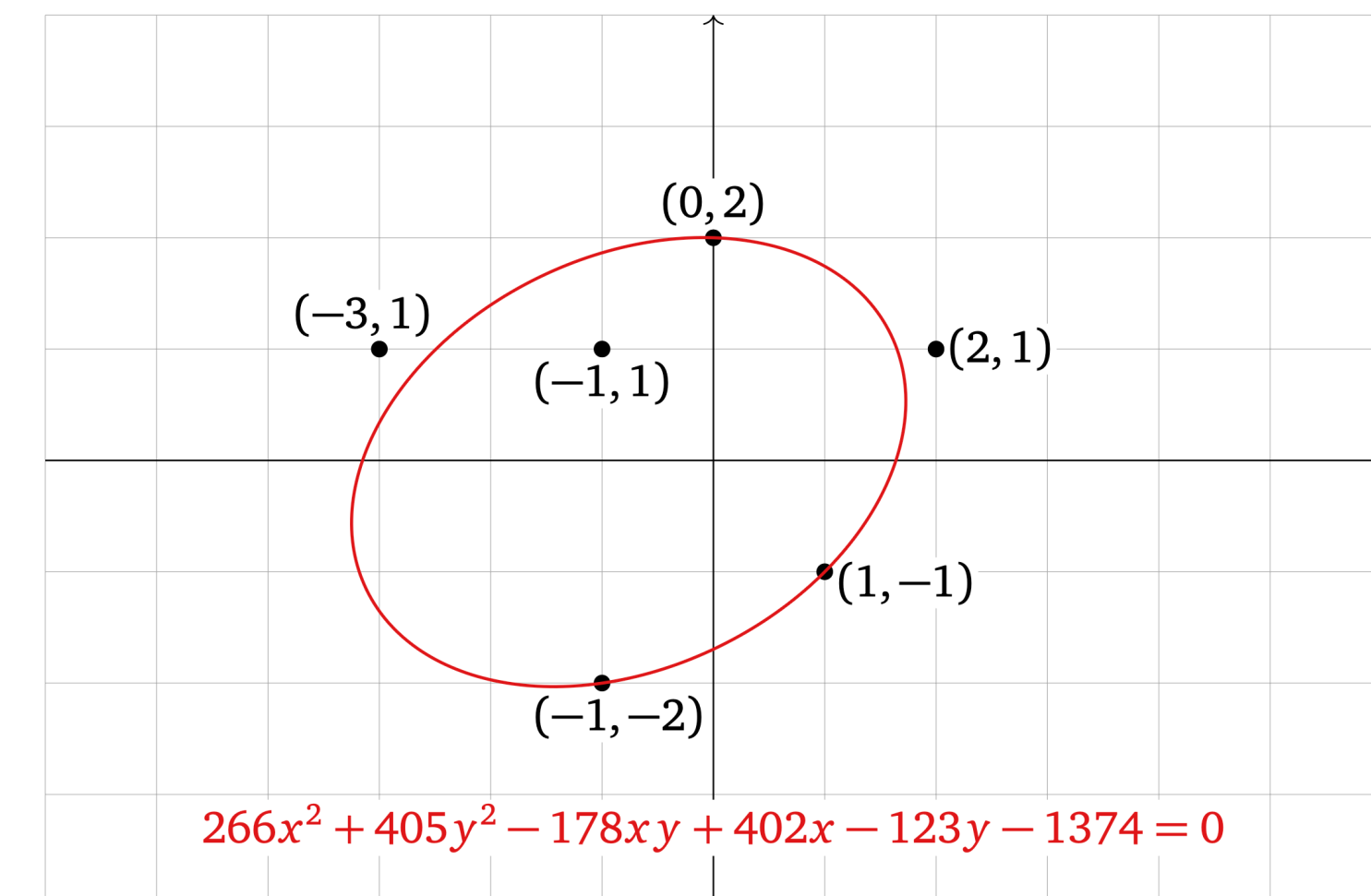
Non-Linearity

Linear algebra is powerful, but
the world isn't linear

There's non-linear stuff



$$b - A\hat{x} = \begin{pmatrix} 6 \\ 0 \\ 0 \end{pmatrix} - A \begin{pmatrix} -3 \\ 5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$$

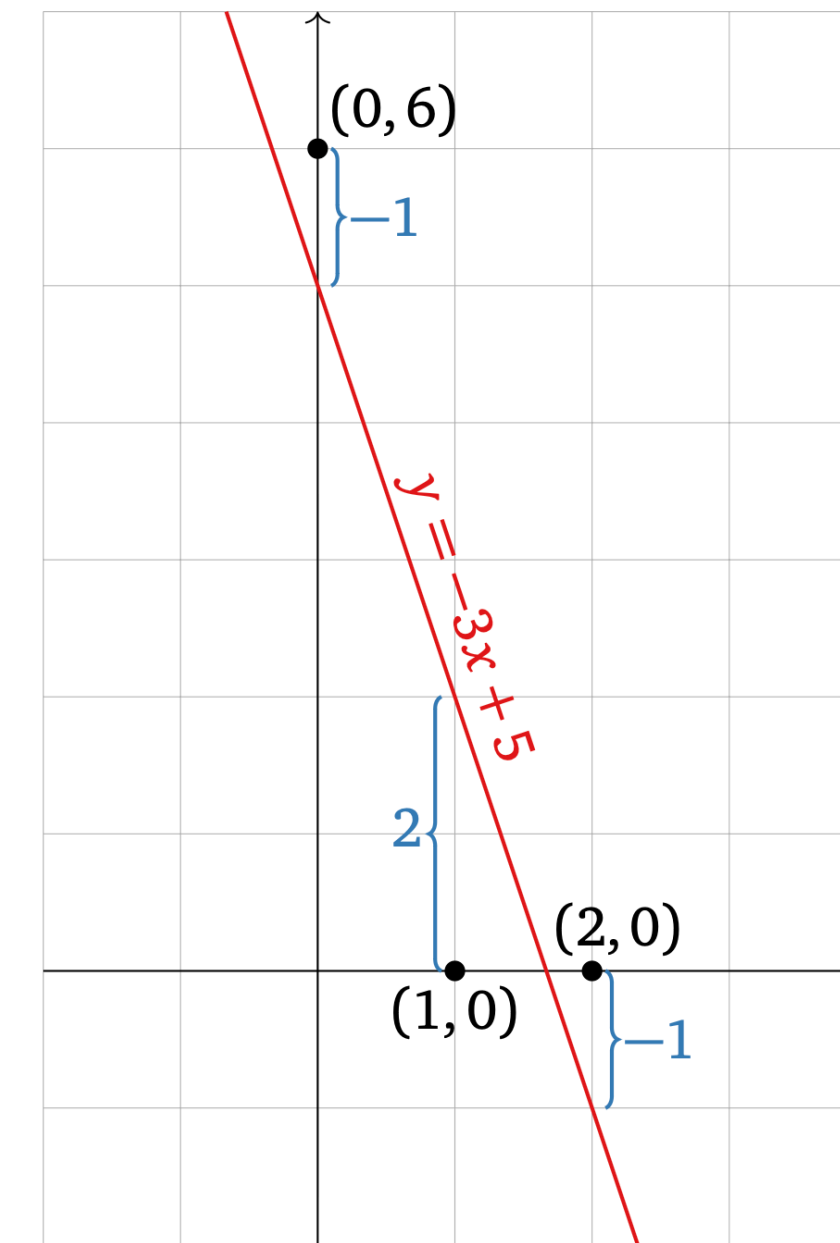


Non-Linearity

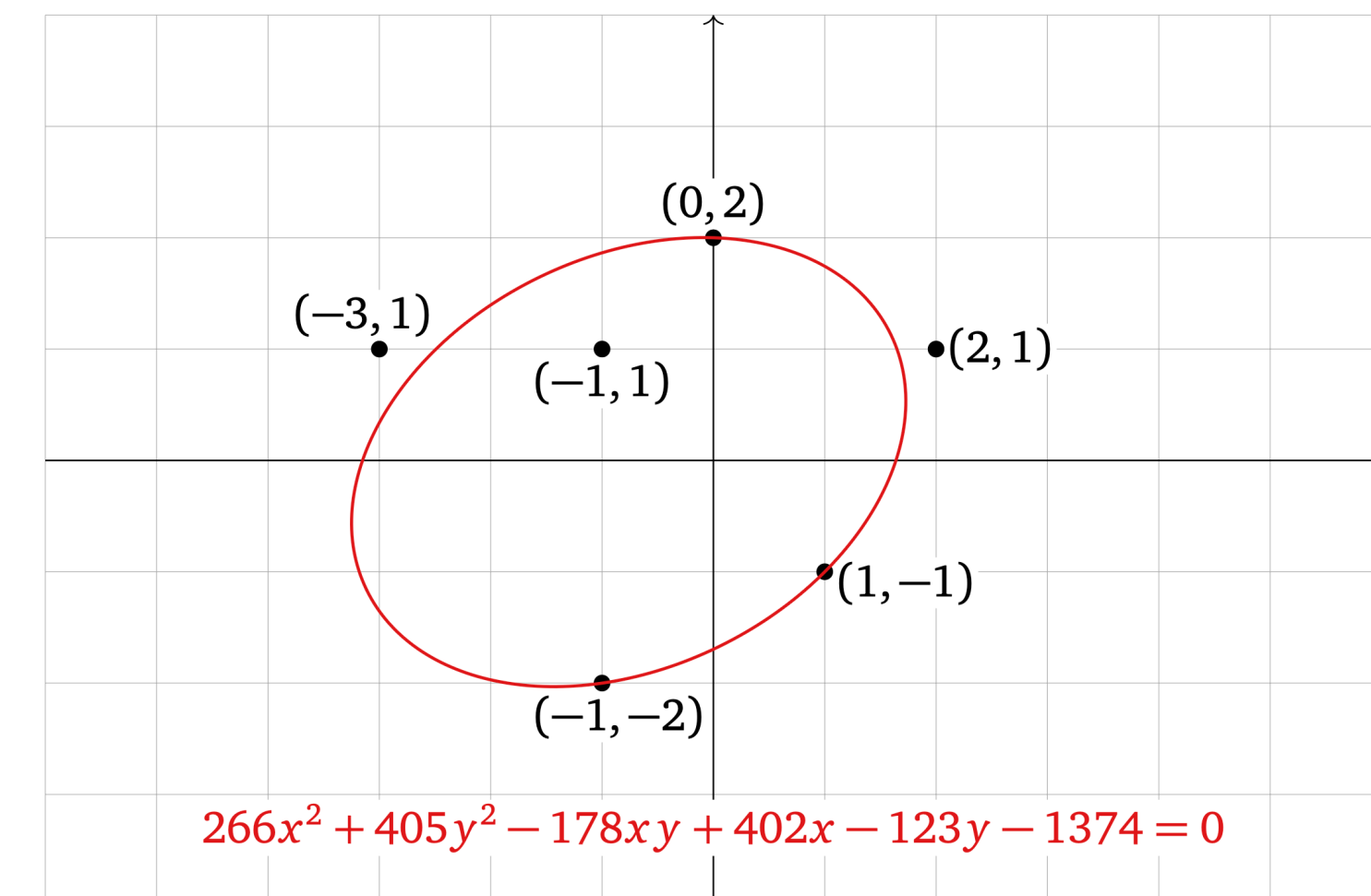
Linear algebra is powerful, but
the world isn't linear

There's non-linear stuff

There's *noise*



$$b - A\hat{x} = \begin{pmatrix} 6 \\ 0 \\ 0 \end{pmatrix} - A \begin{pmatrix} -3 \\ 5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$$



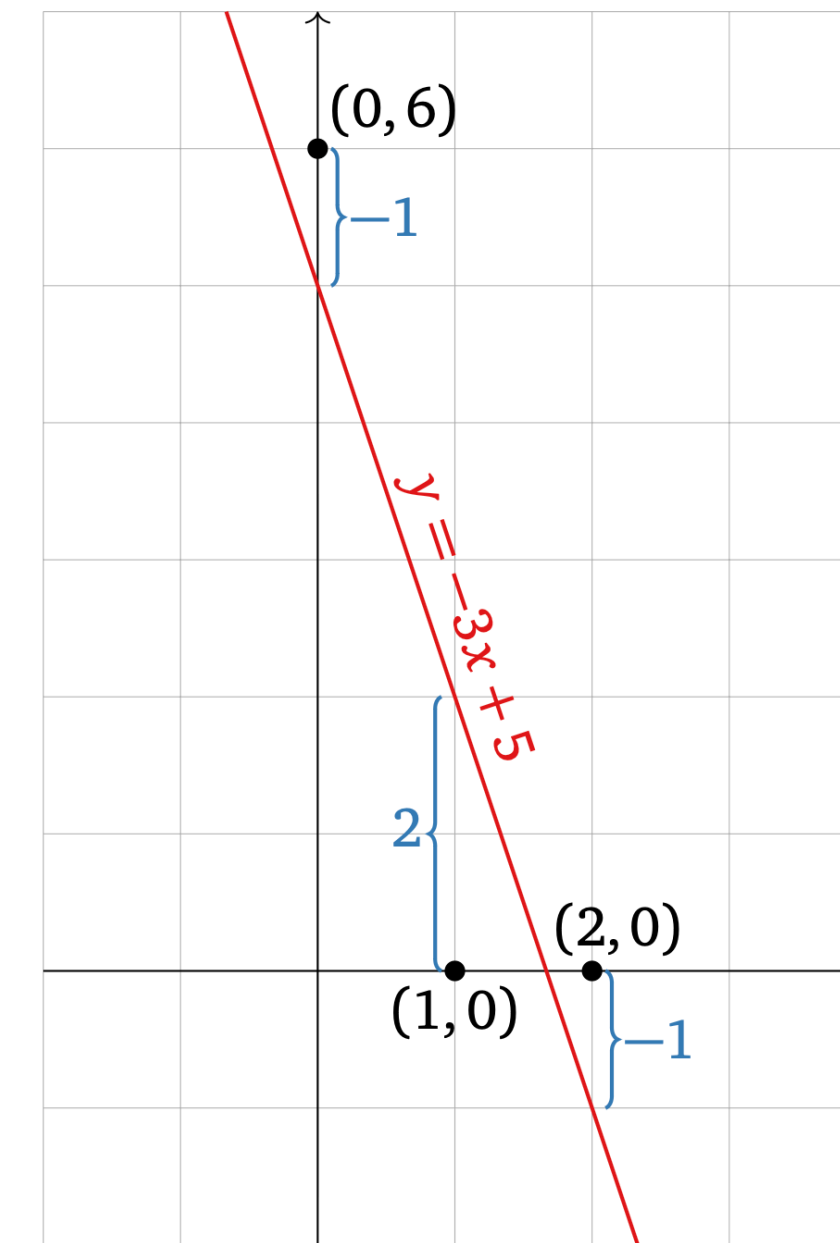
Non-Linearity

Linear algebra is powerful, but
the world isn't linear

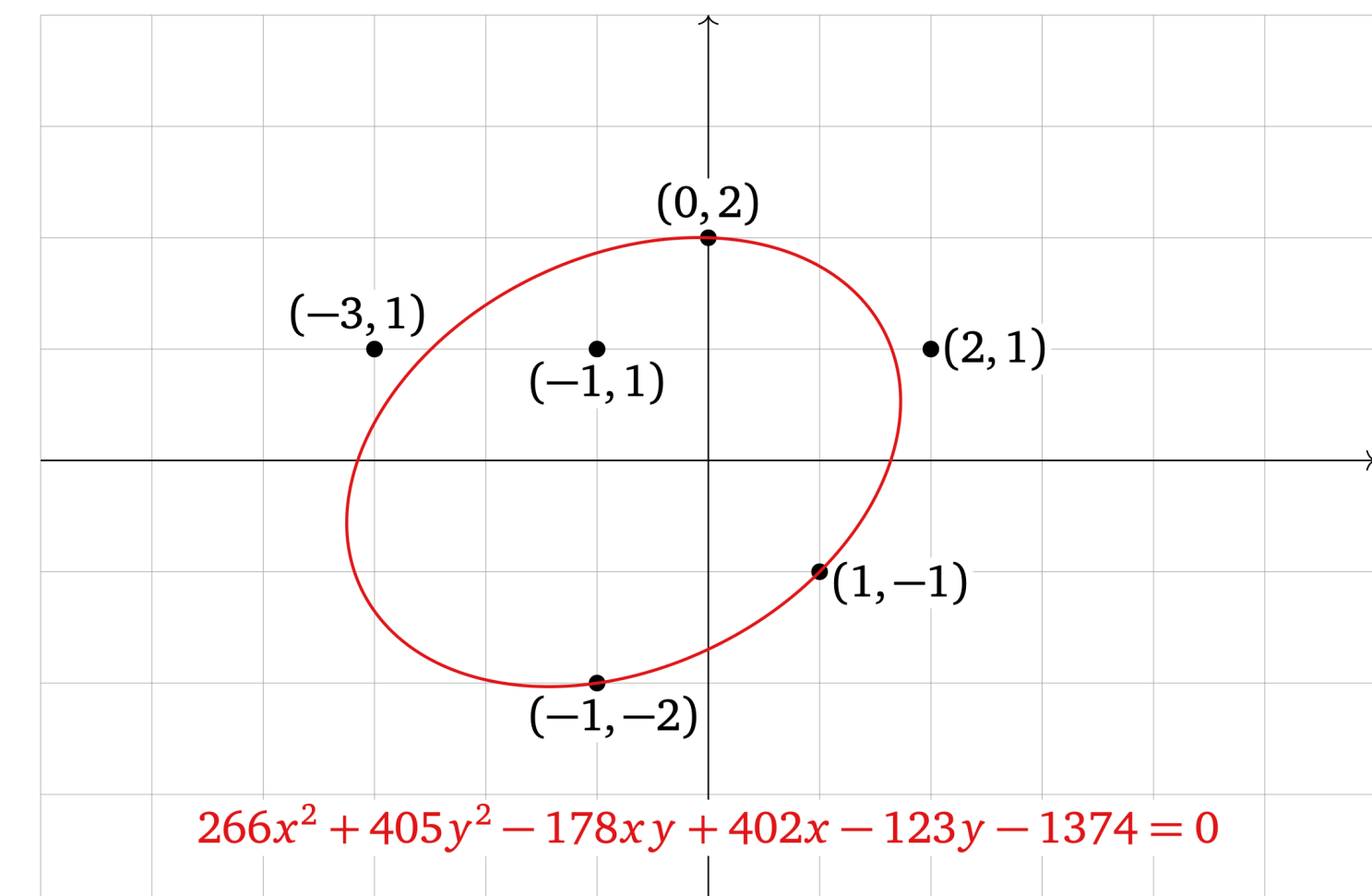
There's non-linear stuff

There's *noise*

We can't make the world linear



$$b - A\hat{x} = \begin{pmatrix} 6 \\ 0 \\ 0 \end{pmatrix} - A \begin{pmatrix} -3 \\ 5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$$



Non-Linearity

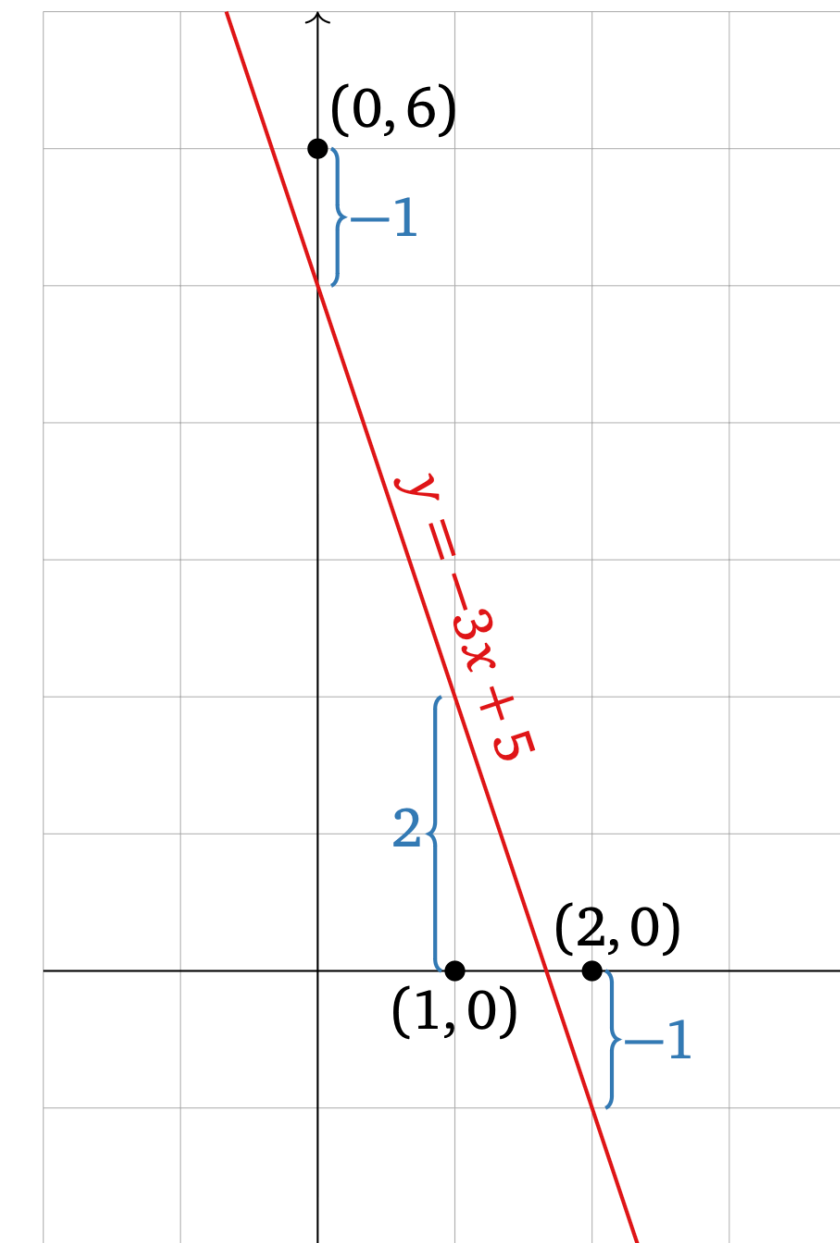
Linear algebra is powerful, but
the world isn't linear

There's non-linear stuff

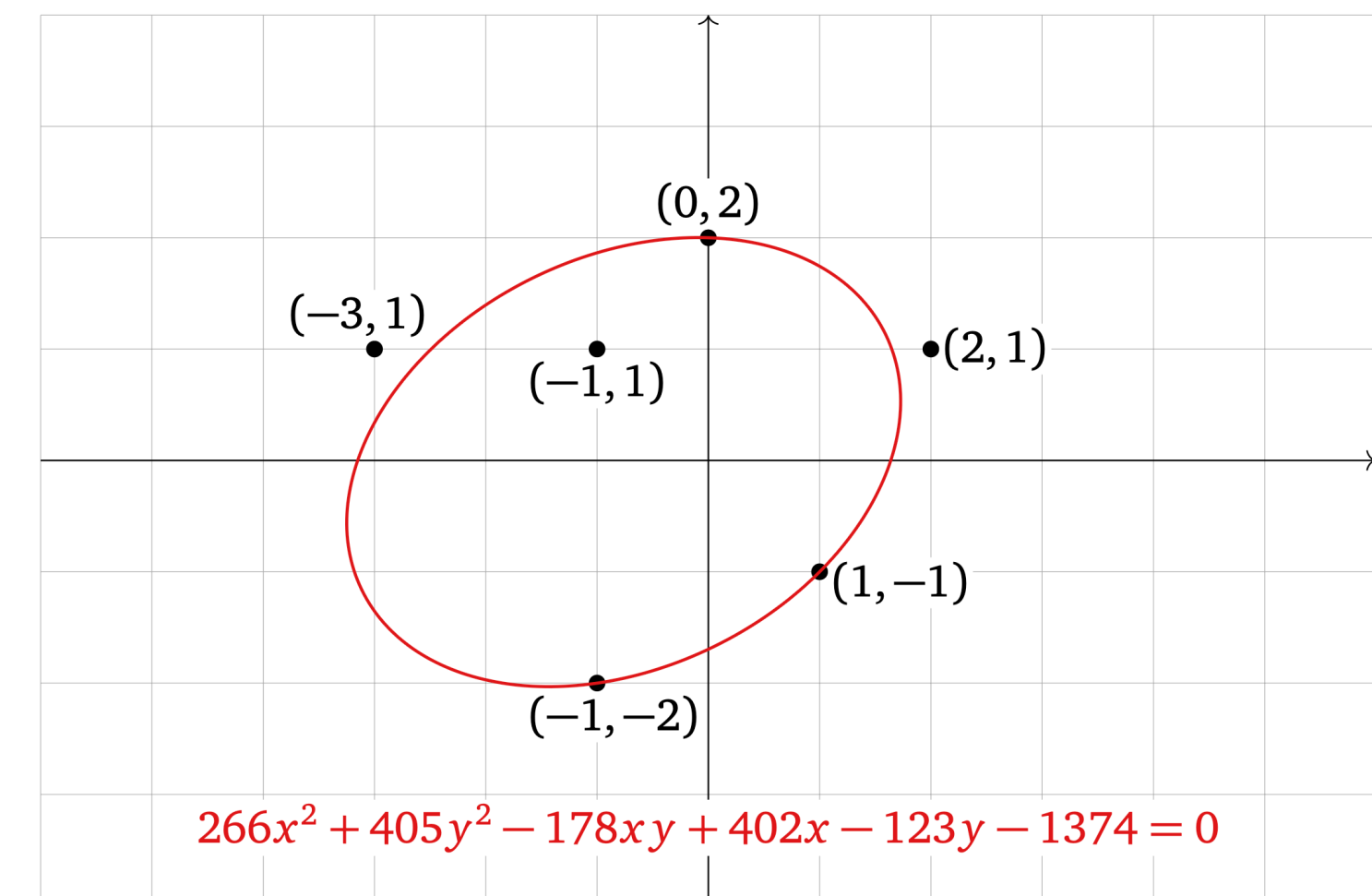
There's *noise*

We can't make the world linear

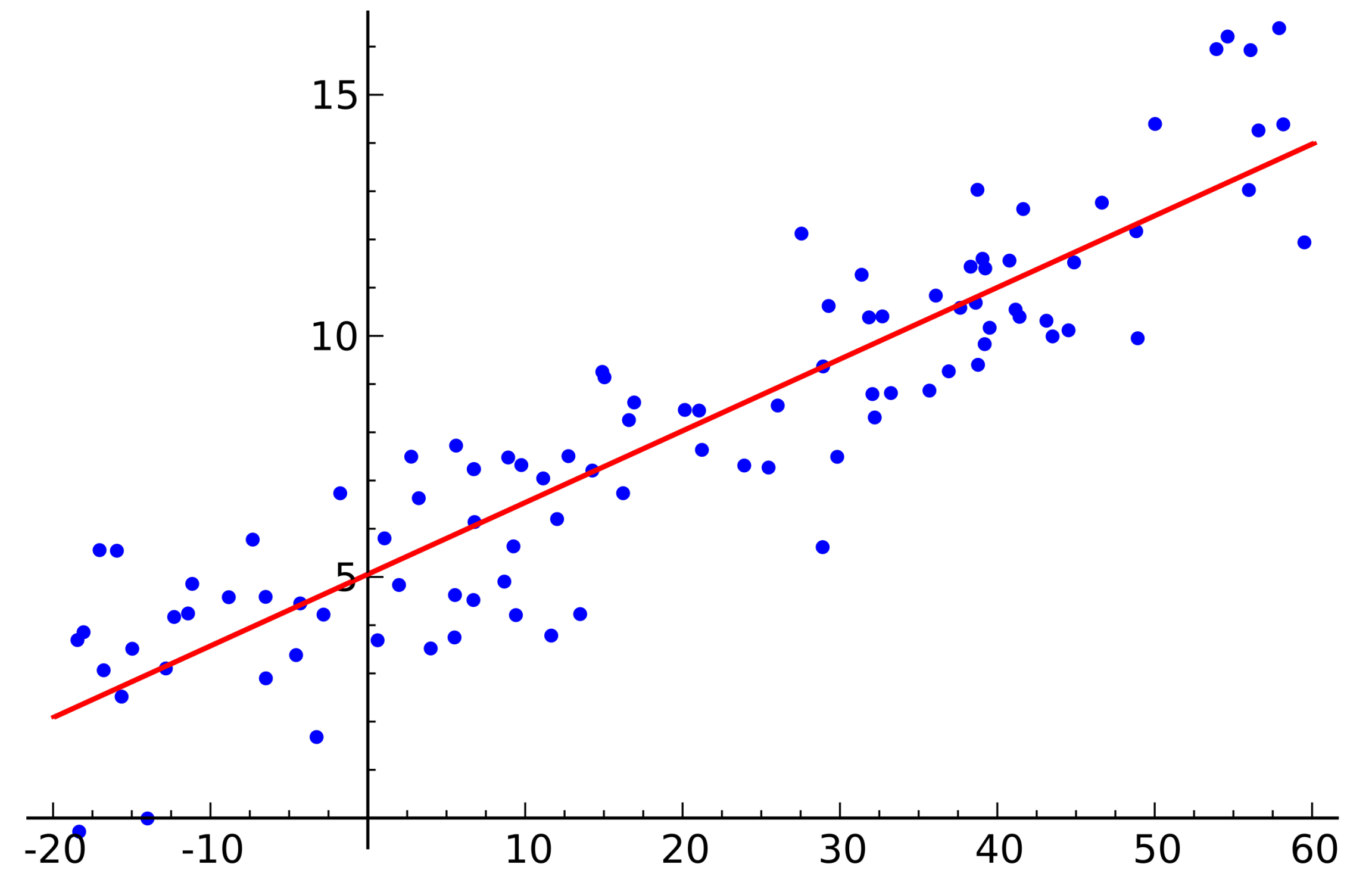
But we can pretend...



$$b - A\hat{x} = \begin{pmatrix} 6 \\ 0 \\ 0 \end{pmatrix} - A \begin{pmatrix} -3 \\ 5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$$

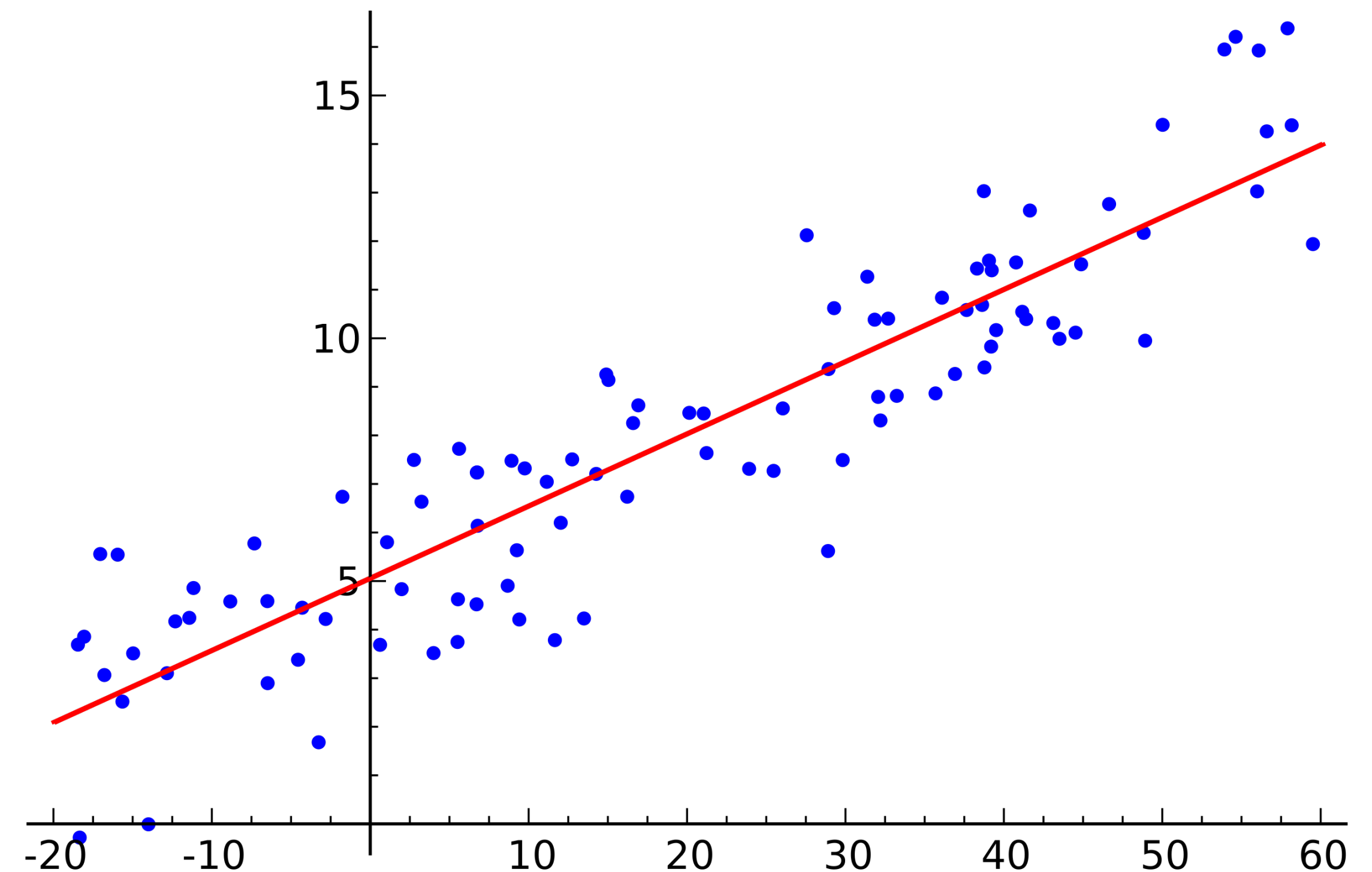


The Idea



The Idea

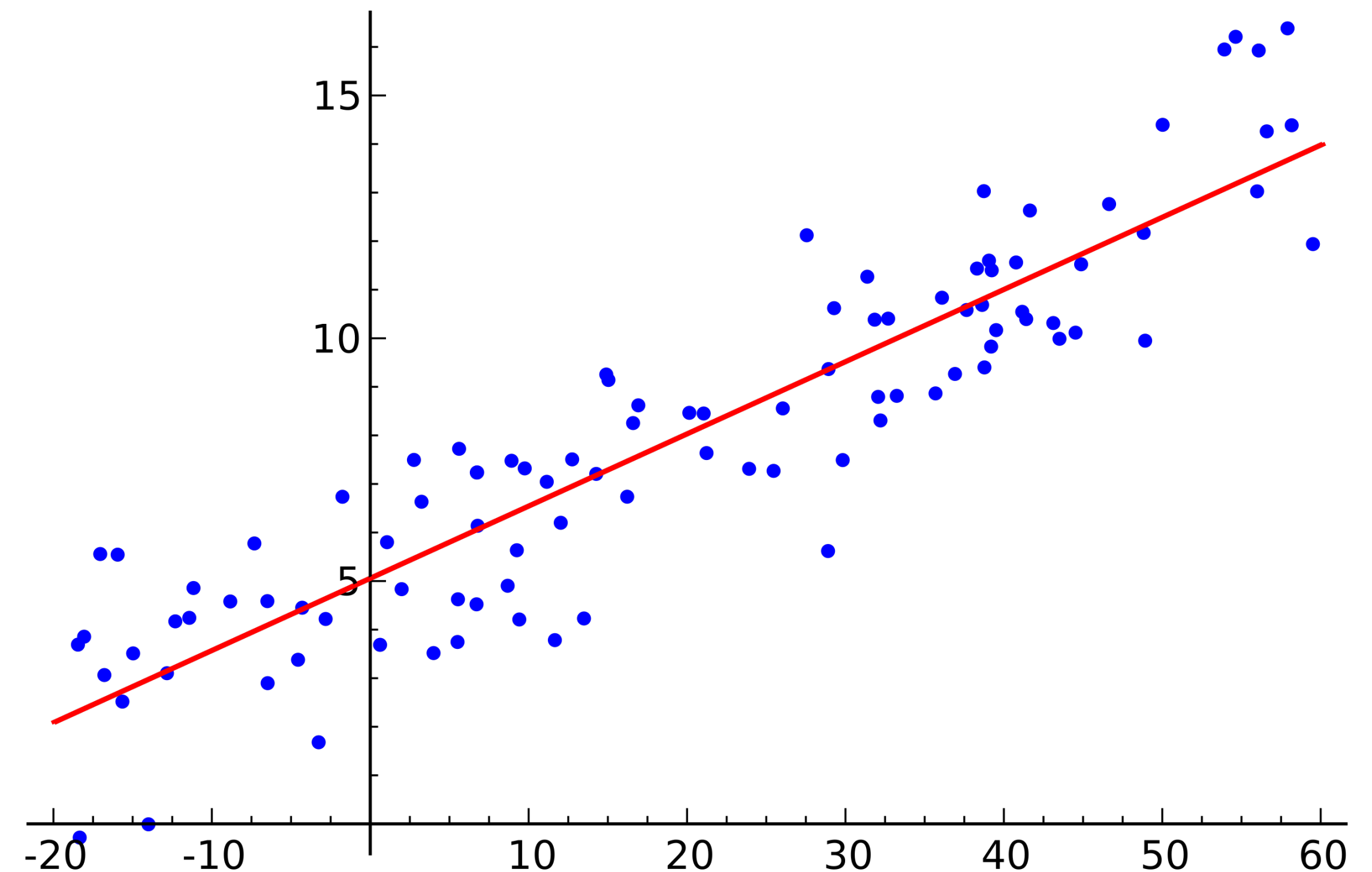
Least Squares is a method for *approximating* solutions to systems of linear equations



The Idea

Least Squares is a method for *approximating* solutions to systems of linear equations

This is **more useful** than exact solutions

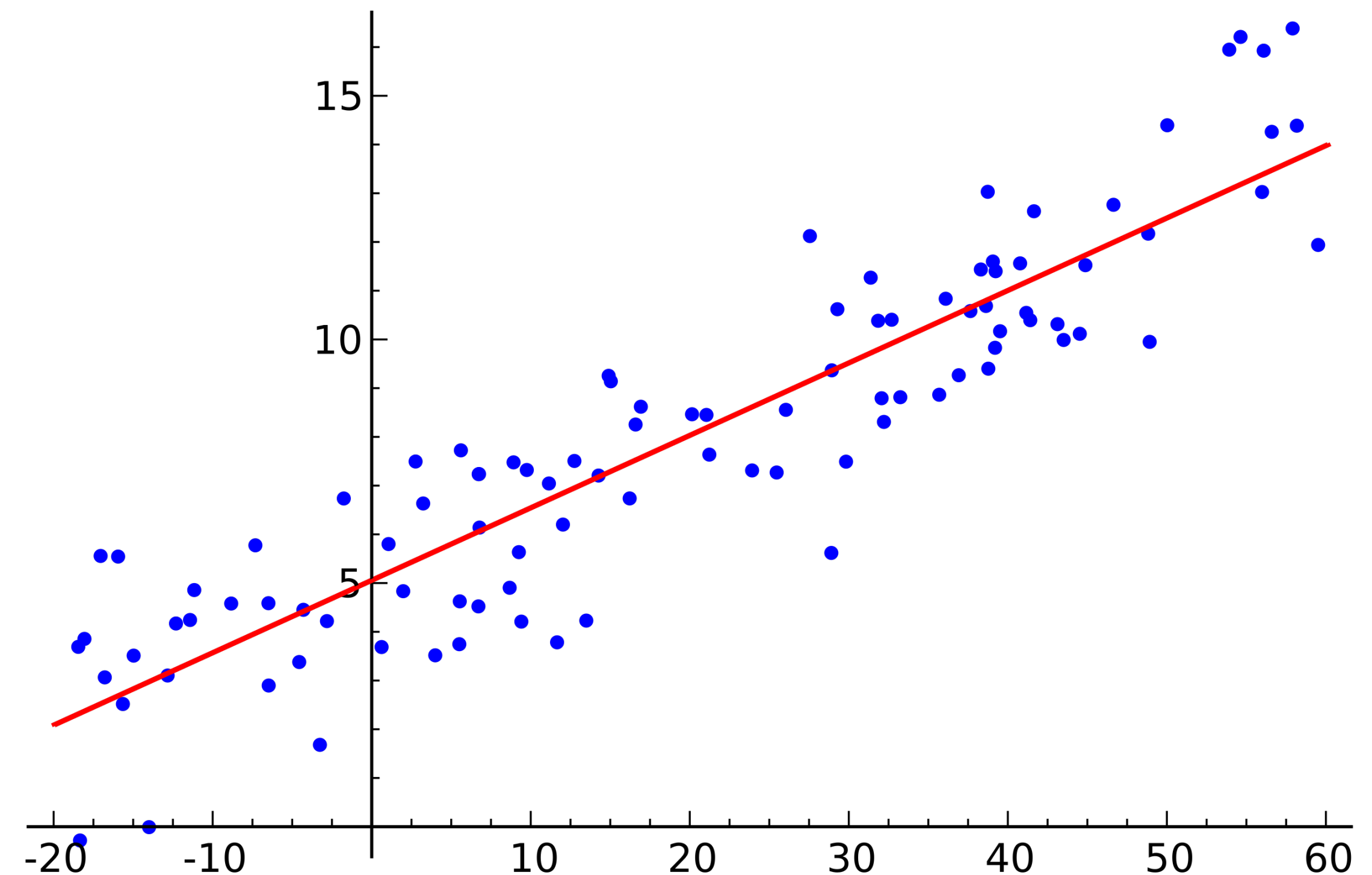


The Idea

Least Squares is a method for *approximating* solutions to systems of linear equations

This is **more useful** than exact solutions

We can do **linear regression**



General Least Squares Problem

The Picture

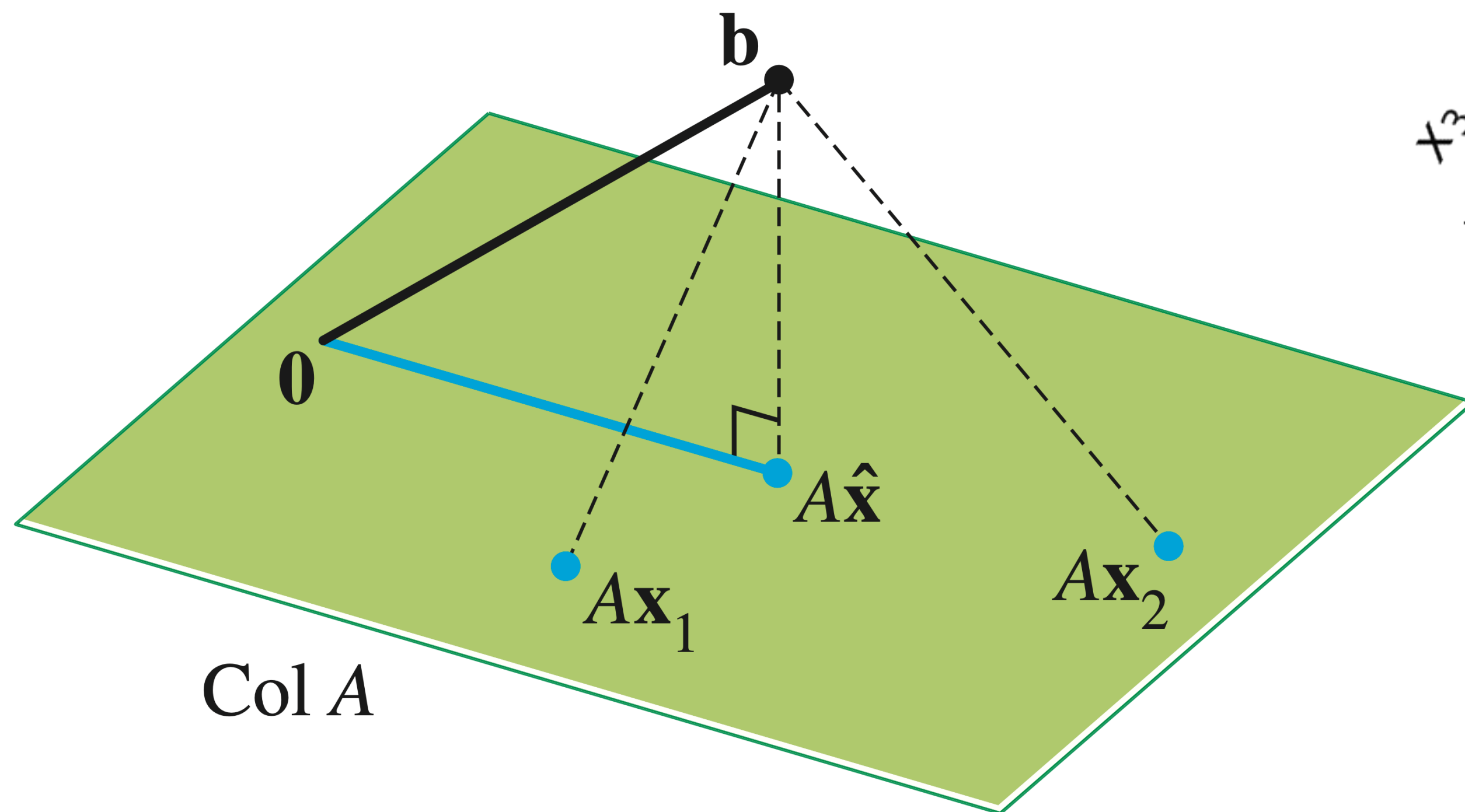
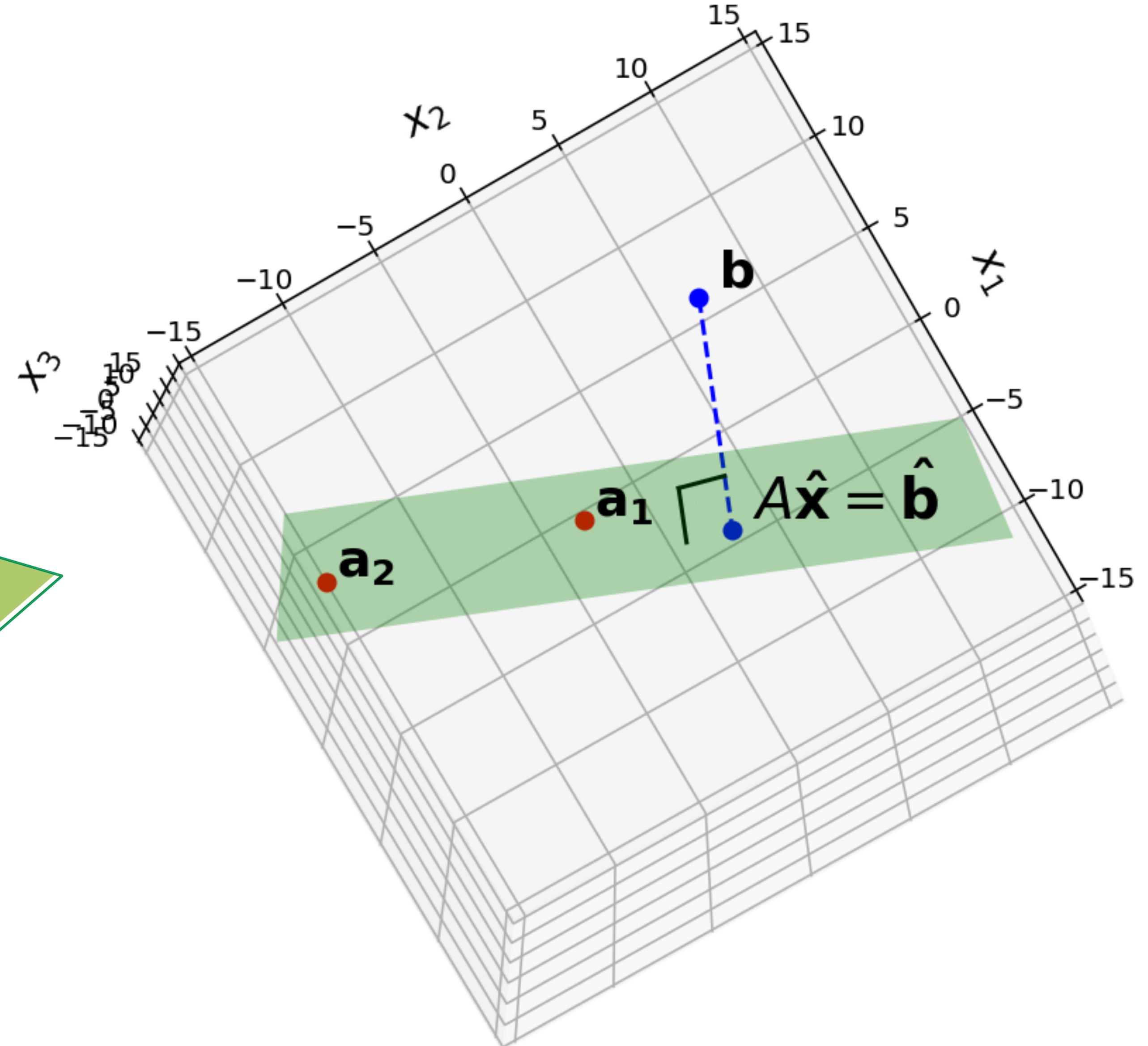


Figure 22.8

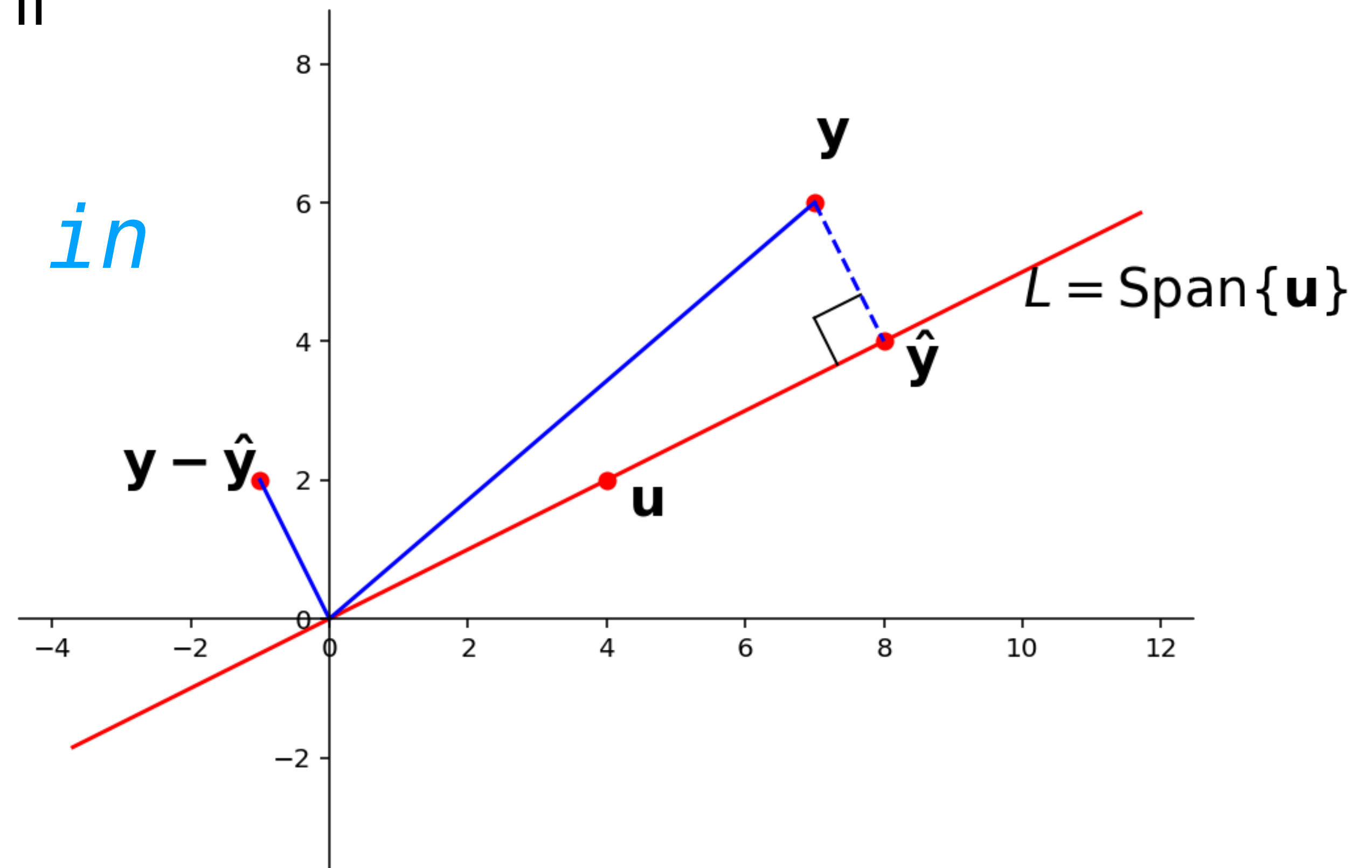
$\hat{\mathbf{b}}$ is closest point in Col A to $\mathbf{b}$



Recall: $\hat{\mathbf{y}}$ and Distance

Thm. $\|\hat{\mathbf{y}} - \mathbf{y}\| = \min_{\mathbf{w} \in \text{span}\{\mathbf{u}\}} \|\mathbf{w} - \mathbf{y}\|$

$\hat{\mathbf{y}}$ is the closest vector in $\text{span}\{\mathbf{u}\}$ to $\mathbf{y}$



The Equational Perspective

The Equational Perspective

We know the equation $x\mathbf{u} = \mathbf{y}$ may have no solution

The Equational Perspective

We know the equation $x\mathbf{u} = \mathbf{y}$ may have no solution

Q. Find α s.t. $\alpha\mathbf{u}$ is as close as possible to $\mathbf{y}$

The Equational Perspective

We know the equation $x\mathbf{u} = \mathbf{y}$ may have no solution

Q. Find α s.t. $\alpha\mathbf{u}$ is as close as possible to $\mathbf{y}$

i.e., $\text{dist}(\mathbf{y}, \alpha\mathbf{u}) = \|\mathbf{y} - \alpha\mathbf{u}\|$ is as small as possible

The Equational Perspective

We know the equation $x\mathbf{u} = \mathbf{y}$ may have no solution

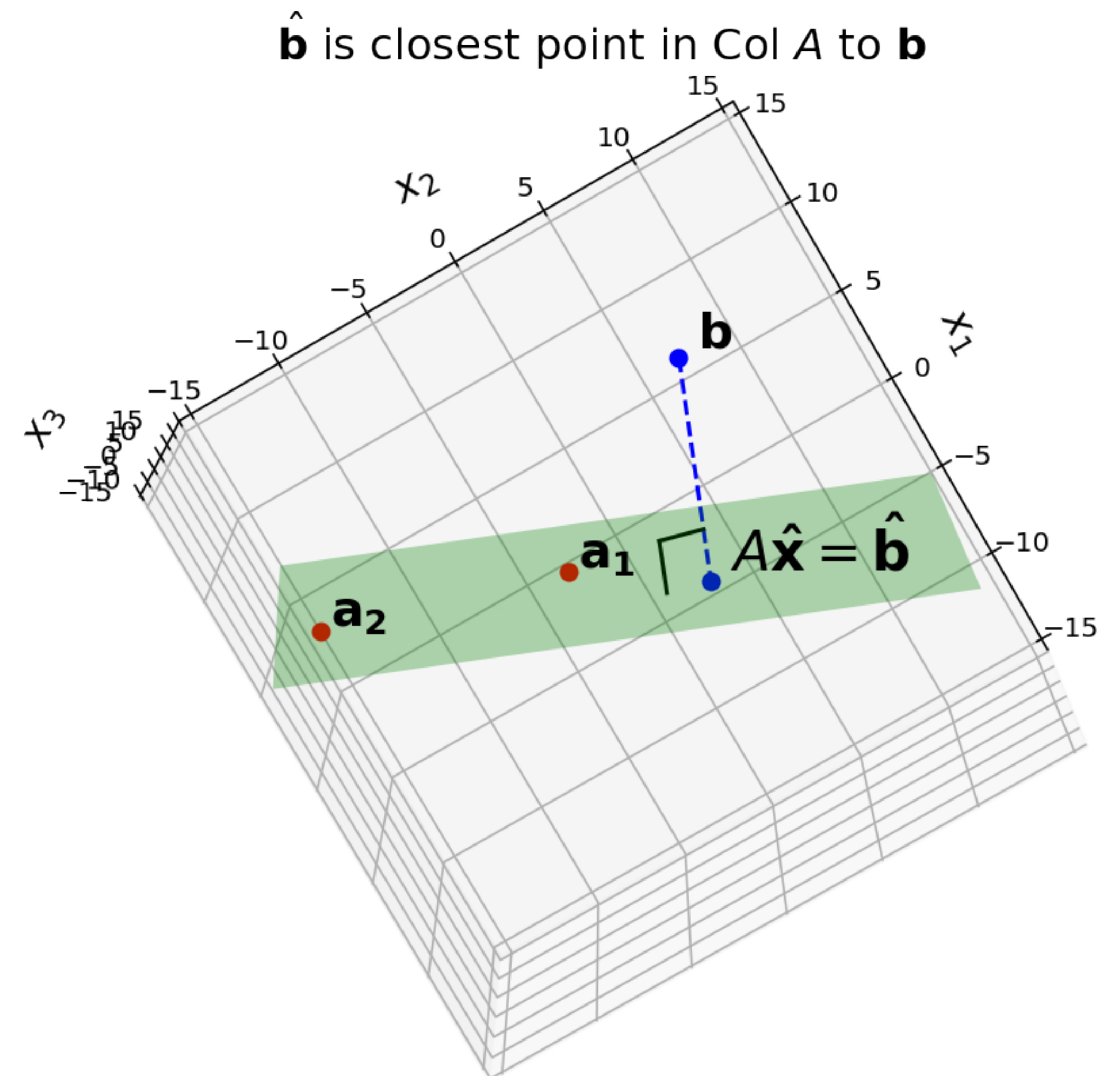
Q. Find α s.t. $\alpha\mathbf{u}$ is as close as possible to $\mathbf{y}$

i.e., $\text{dist}(\mathbf{y}, \alpha\mathbf{u}) = \|\mathbf{y} - \alpha\mathbf{u}\|$ is as small as possible

We need to generalize this

The General Least Squares Problem

Figure 22.8

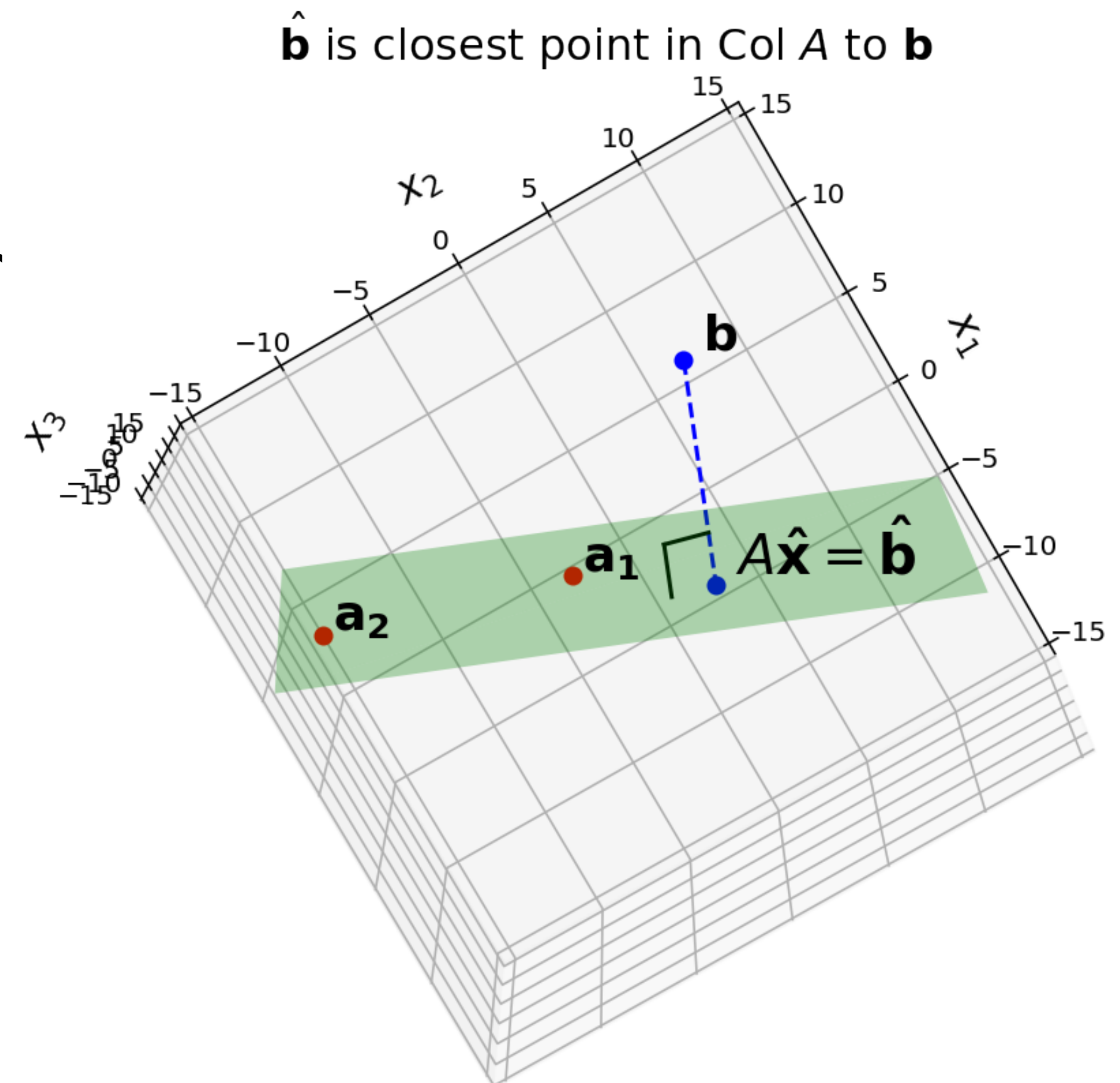


The General Least Squares Problem

Figure 22.8

Q. Given $A \in \mathbb{R}^{m \times n}$ and a vector $\mathbf{b} \in \mathbb{R}^m$, find a vector $\mathbf{x} \in \mathbb{R}^n$ that *minimizes*

$$\text{dist}(A\mathbf{x}, \mathbf{b}) = \|A\mathbf{x} - \mathbf{b}\|$$



The General Least Squares Problem

Figure 22.8

Q. Given $A \in \mathbb{R}^{m \times n}$ and a vector $\mathbf{b} \in \mathbb{R}^m$, find a vector $\mathbf{x} \in \mathbb{R}^n$ that *minimizes*

$$\text{dist}(A\mathbf{x}, \mathbf{b}) = \|A\mathbf{x} - \mathbf{b}\|$$

Find a vector $\mathbf{x}$ which makes $\|A\mathbf{x} - \mathbf{b}\|$ as small as possible

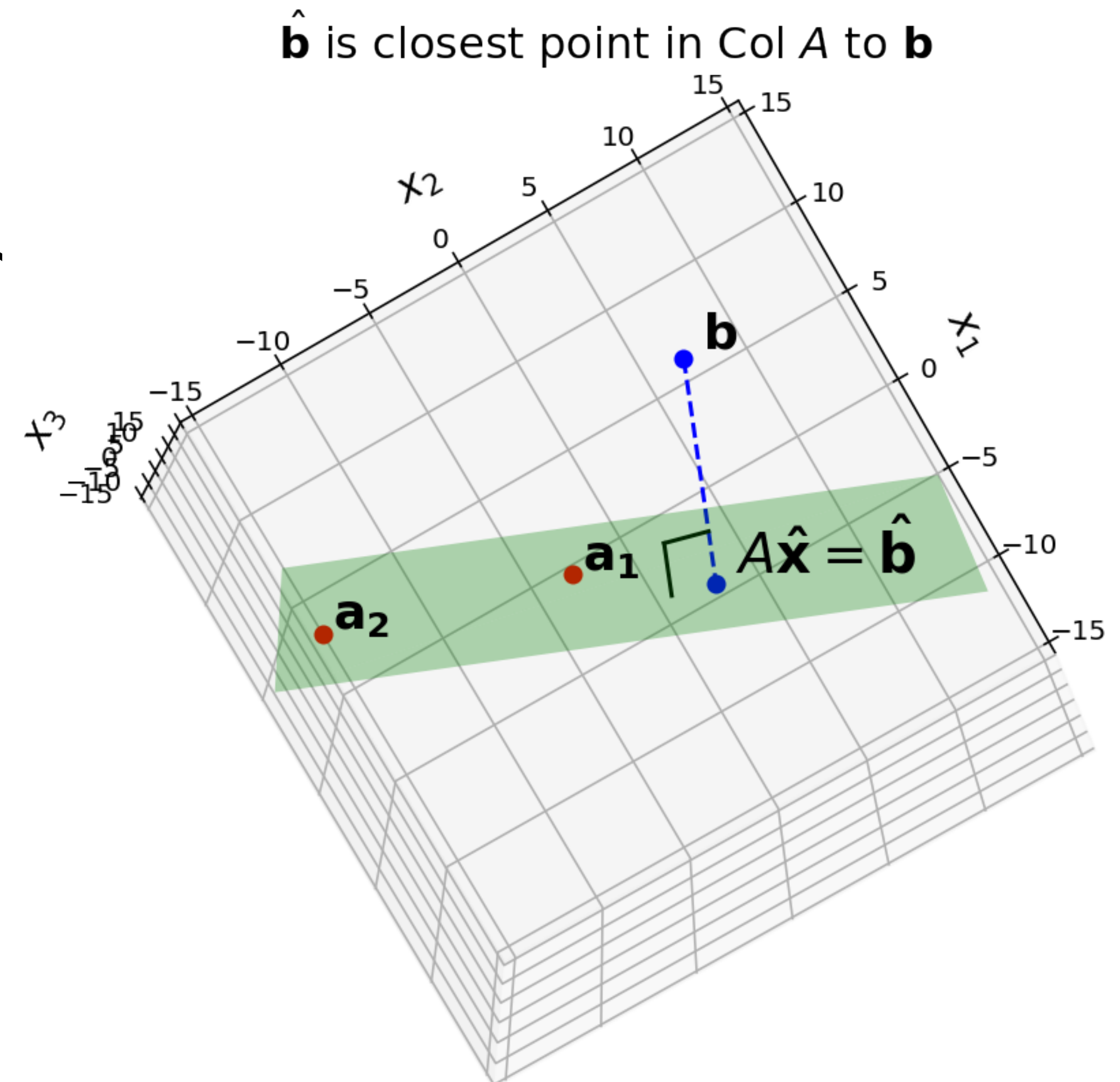
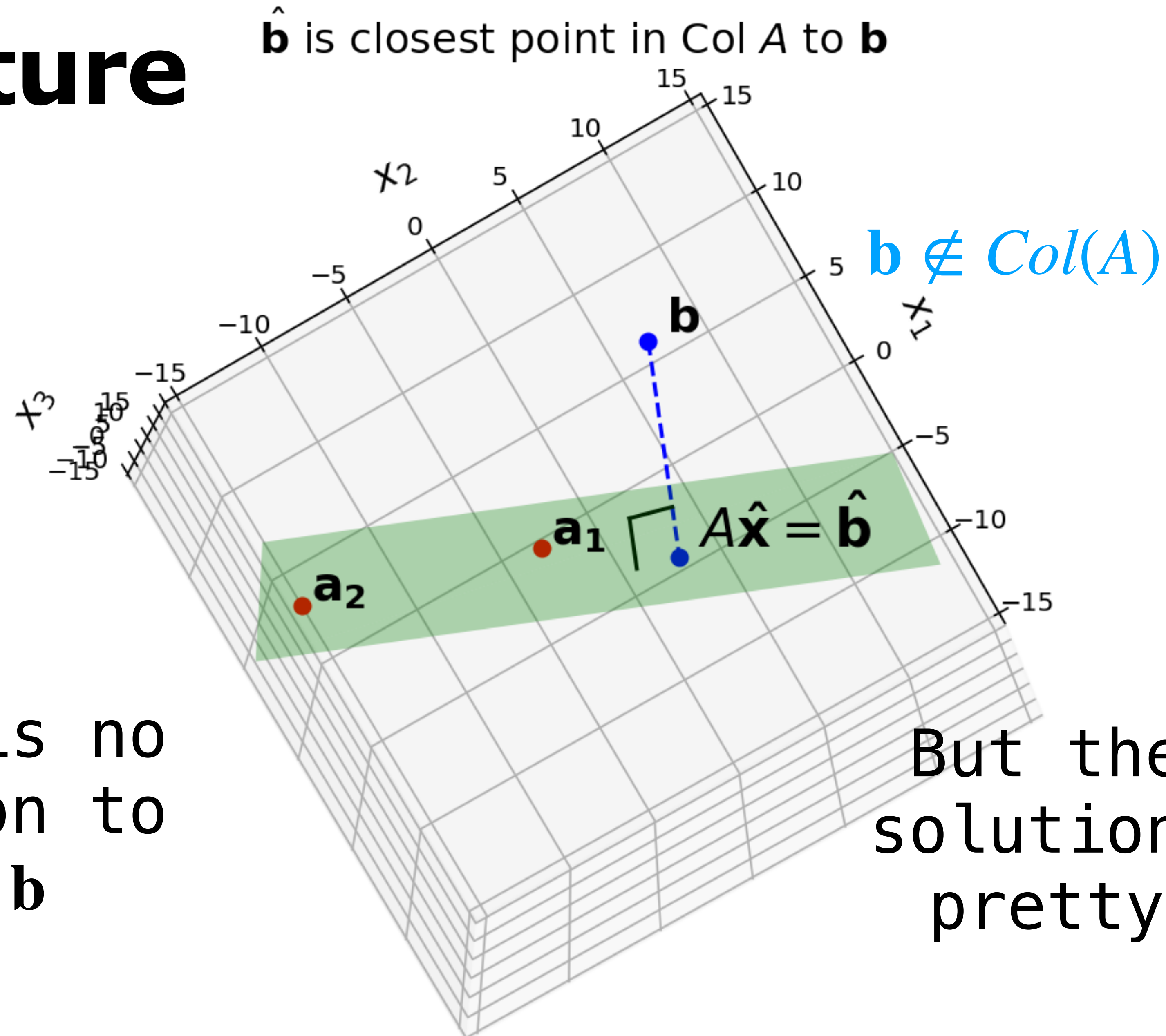


Figure 22.8

The Picture



There is no
solution to
 $A\mathbf{x} = \mathbf{b}$

But there's a
solution that's
pretty close

Sum of Squares

$$\|A\mathbf{x} - \mathbf{b}\|^2 = \sum_{i=1}^n ((A\mathbf{x})_i - \mathbf{b}_i)^2$$

Sum of Squares

$$\|A\mathbf{x} - \mathbf{b}\|^2 = \sum_{i=1}^n ((A\mathbf{x})_i - \mathbf{b}_i)^2$$

It's equivalent to minimize $\|A\mathbf{x} - \mathbf{b}\|^2$

Sum of Squares

$$\|A\mathbf{x} - \mathbf{b}\|^2 = \sum_{i=1}^n ((A\mathbf{x})_i - \mathbf{b}_i)^2$$

It's equivalent to minimize $\|A\mathbf{x} - \mathbf{b}\|^2$

Sums of squares come up everywhere

Sum of Squares

$$\|A\mathbf{x} - \mathbf{b}\|^2 = \sum_{i=1}^n ((A\mathbf{x})_i - \mathbf{b}_i)^2$$

It's equivalent to minimize $\|A\mathbf{x} - \mathbf{b}\|^2$

Sums of squares come up everywhere

Note. This error is everywhere differentiable,

whereas $\sum_{i=1}^n |(A\mathbf{x})_i - b_i|$ is not

Least Squares Solution

Def. Given $A \in \mathbb{R}^{m \times n}$ and a vector $\mathbf{b} \in \mathbb{R}^m$, a **least squares solution** of $A\mathbf{x} = \mathbf{b}$ is a vector $\hat{\mathbf{x}} \in \mathbb{R}^n$ s.t.

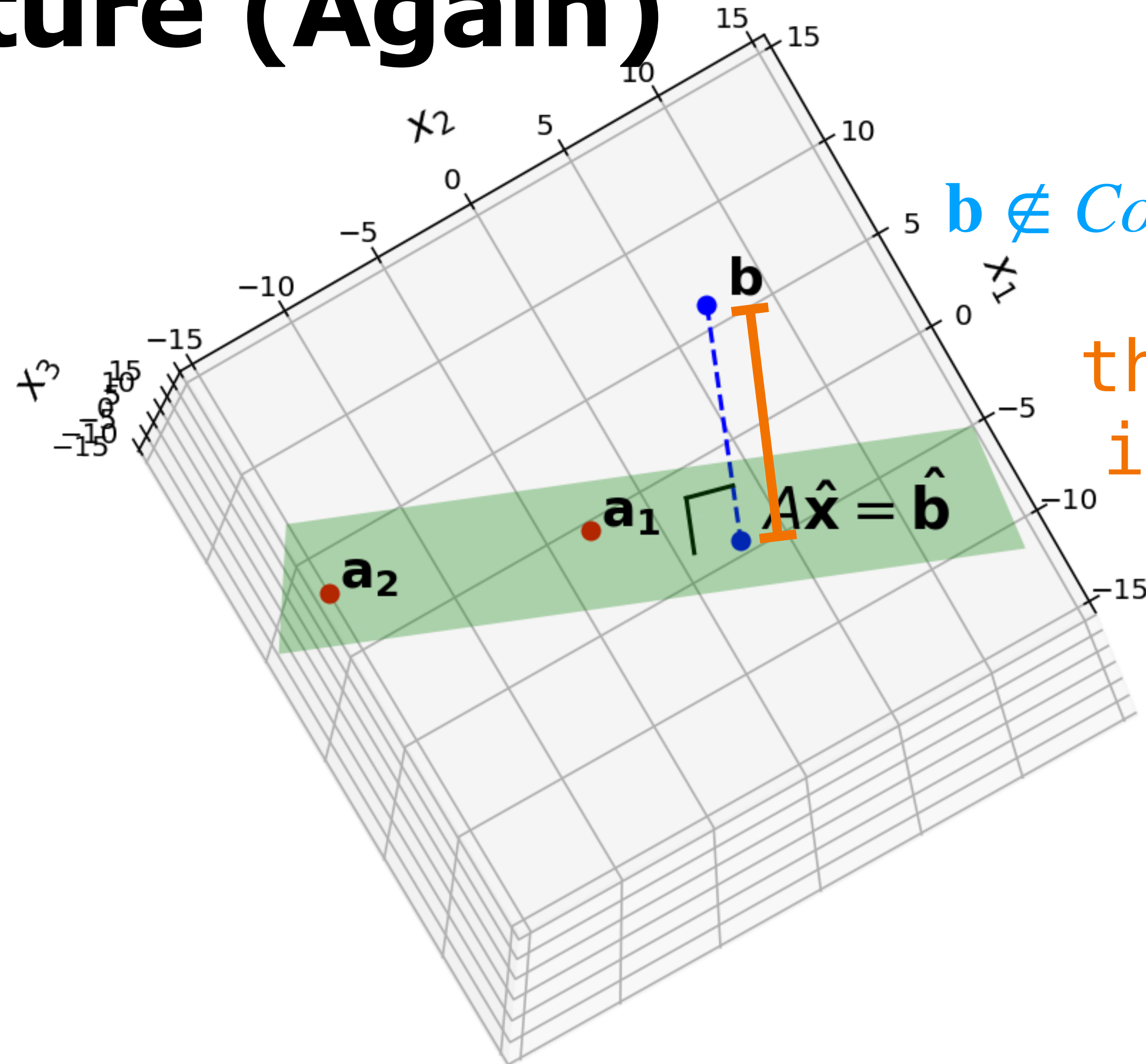
$$\|A\hat{\mathbf{x}} - \mathbf{b}\| \leq \|A\mathbf{x} - \mathbf{b}\|$$

for *any* $\mathbf{x}$ in $\mathbb{R}^n$

Again, $\|A\hat{\mathbf{x}} - \mathbf{b}\|$ is as small as possible

Figure 22.8

The Picture (Again)



$\mathbf{b} \notin \text{Col}(A)$

this distance
is minimized

Argmin

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|$$

Argmin

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|$$

Another way of framing this is via arg min

Argmin

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|$$

Another way of framing this is via arg min

Def. $\arg \min_{x \in X} f(x) = \hat{x}$ where $f(\hat{x}) = \min_{x \in X} f(x)$

Argmin

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|$$

Another way of framing this is via arg min

Def. $\arg \min_{x \in X} f(x) = \hat{x}$ where $f(\hat{x}) = \min_{x \in X} f(x)$

$\hat{x}$ is the *argument* that *minimizes* f

Argmin

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|$$

Another way of framing this is via arg min

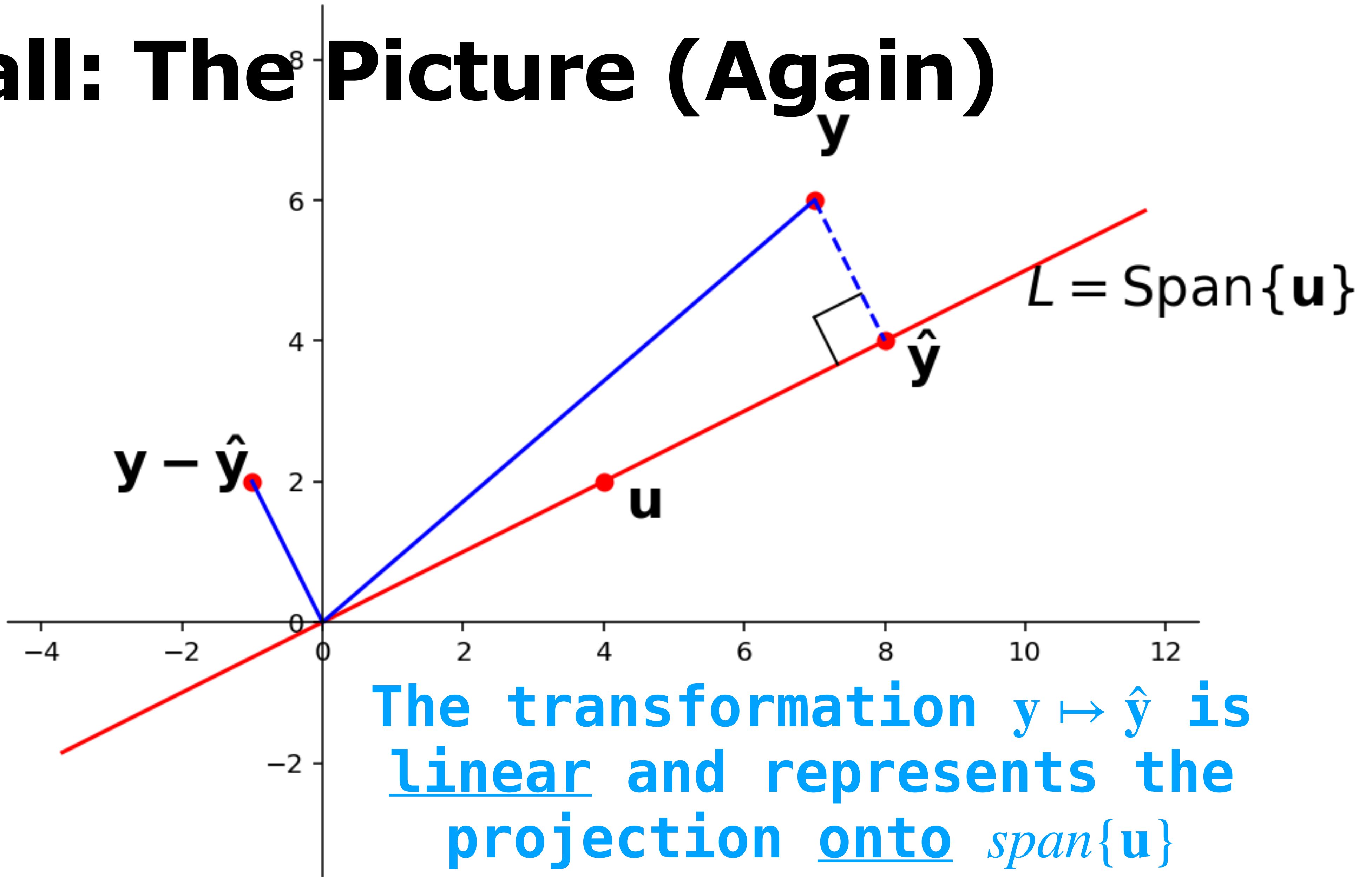
Def. $\arg \min_{x \in X} f(x) = \hat{x}$ where $f(\hat{x}) = \min_{x \in X} f(x)$

$\hat{x}$ is the ***argument*** that ***minimizes*** f

This is now an *optimization problem*

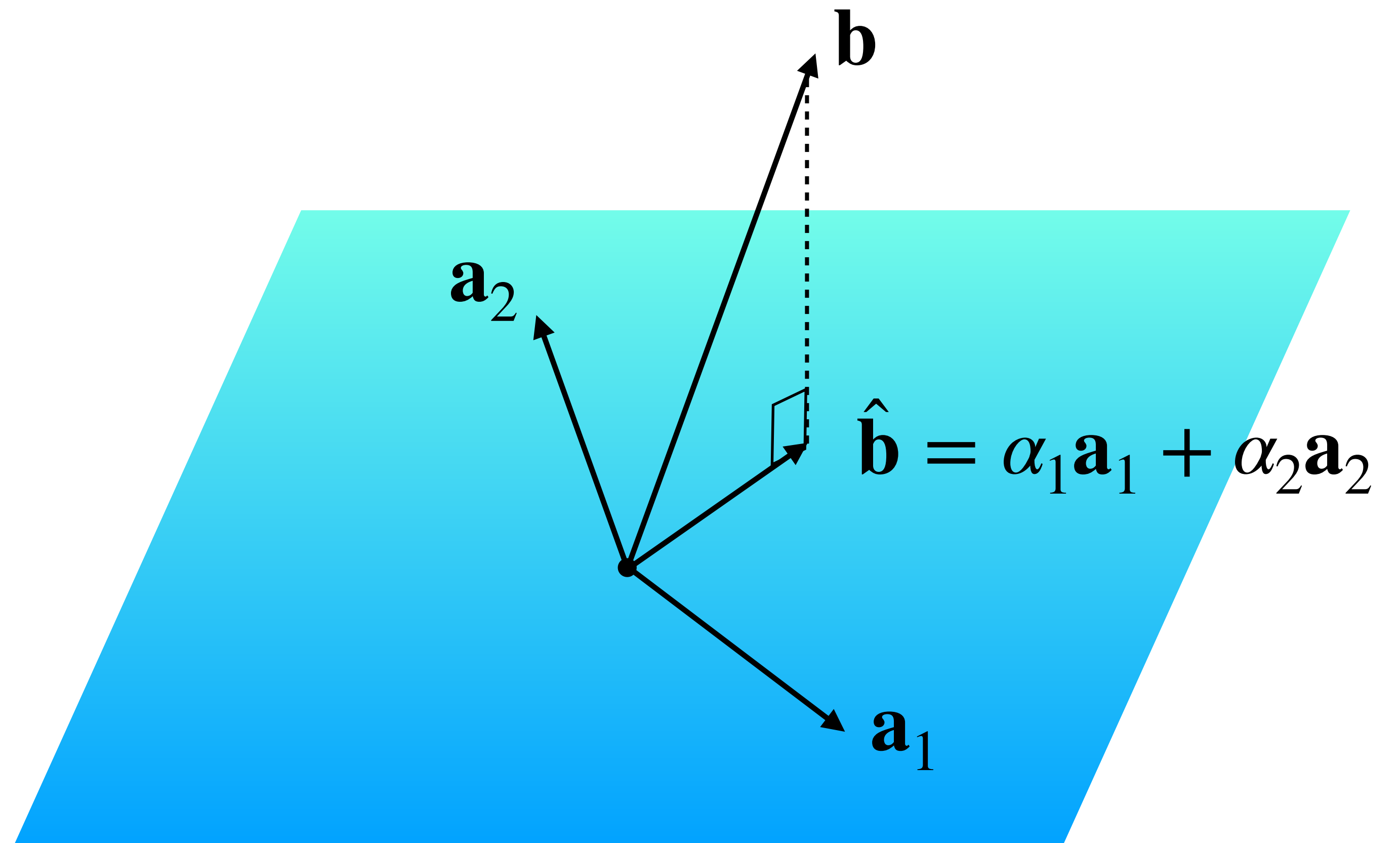
Solving the General Least Squares Problems

Recall: The⁸ Picture (Again)_y



Projects onto other Spans

The transformation
 $\mathbf{b} \mapsto \hat{\mathbf{b}}$ is the
projection of $\mathbf{b}$
onto $\text{span}\{\mathbf{a}_1, \mathbf{a}_2\}$



The High Level Approach.

Q. Find a least squares solutions to $A = \mathbf{b}$

A.

1. Find the closest point $\hat{\mathbf{b}}$ in $Col(A)$ to $\mathbf{b}$

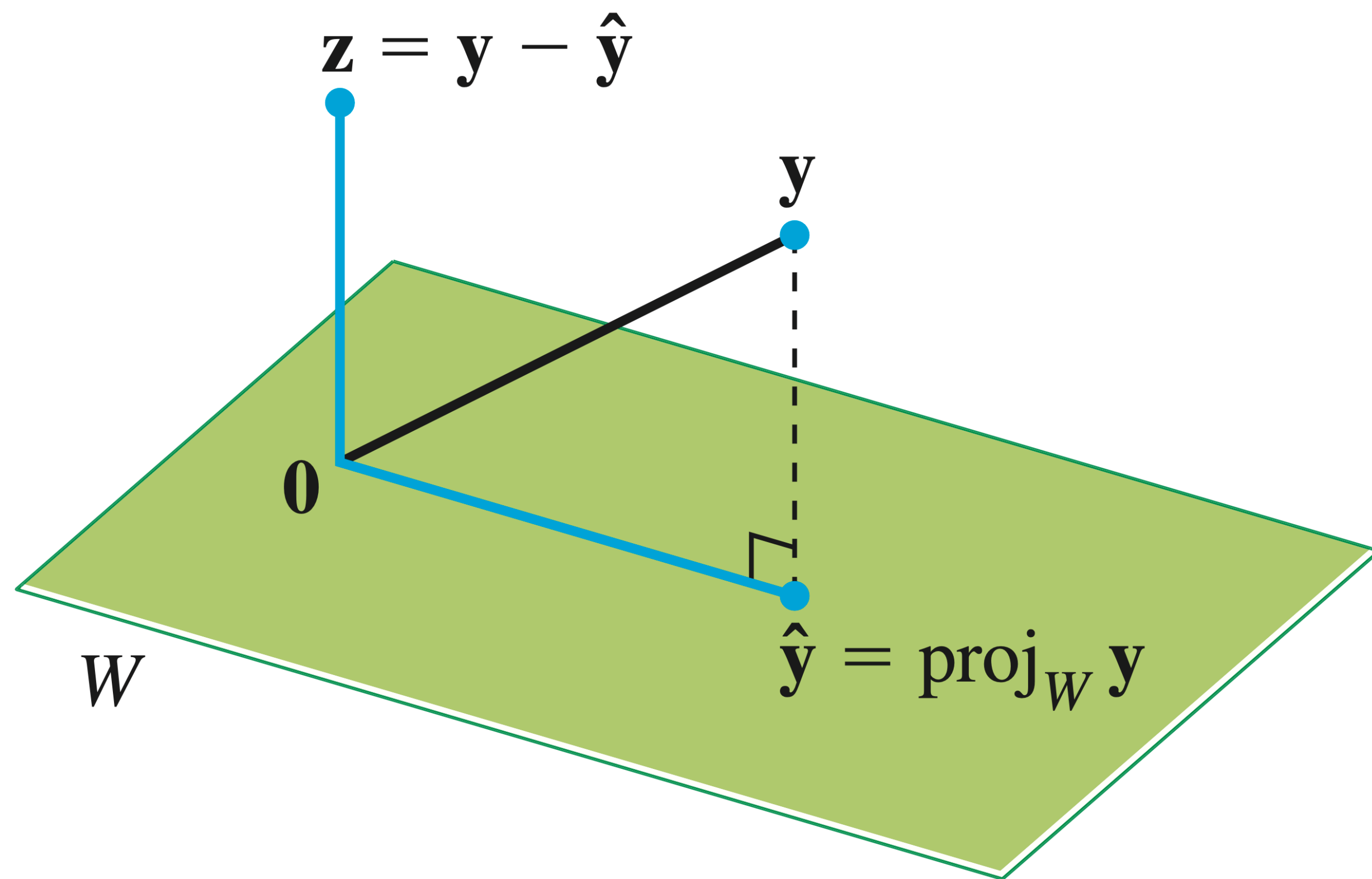
2. Solve the equation $A\mathbf{x} = \hat{\mathbf{b}}$ instead

Orthogonal Decomposition Theorem

Thm. Let W be a subspace of $\mathbb{R}^n$. Every vector y in $\mathbb{R}^n$ can be written *uniquely* as

$$y = \hat{y} + z$$

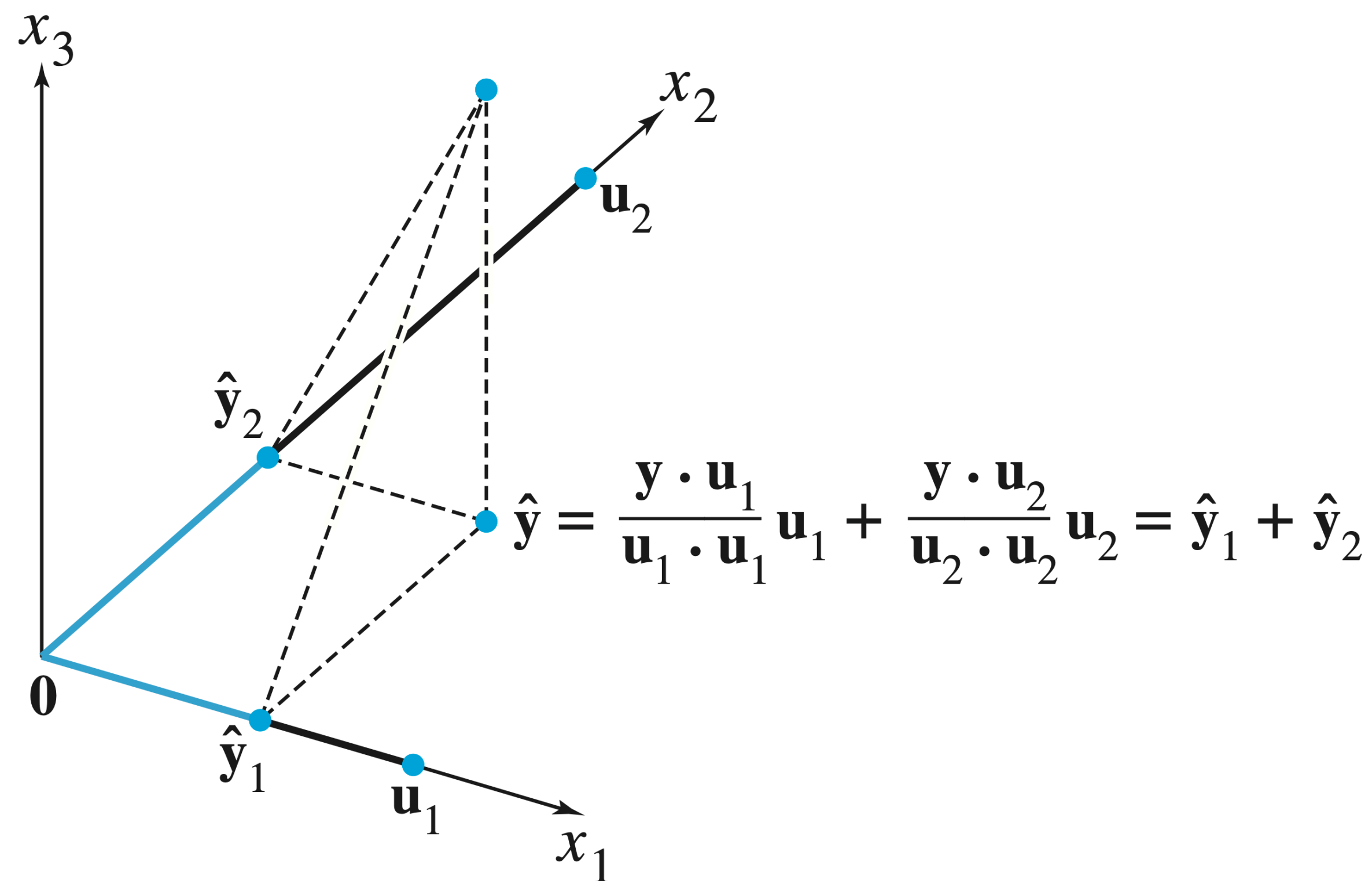
where $\hat{y} \in W$ and z is orthogonal to every vector in W



Projection via Orthogonal Bases

We can determine $\hat{y}$ by projecting onto an orthogonal basis

Every subspace has an orthogonal basis (we won't prove this)



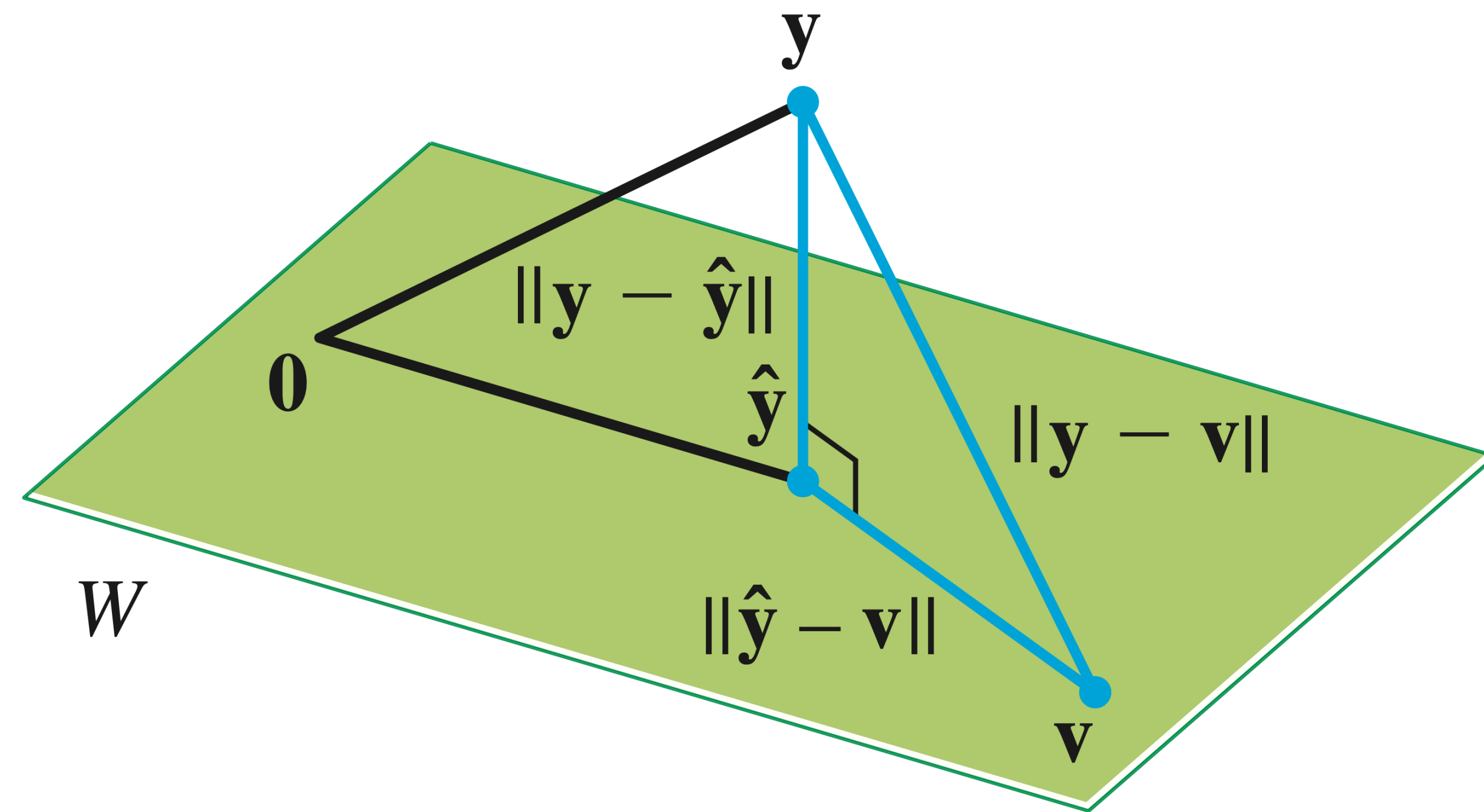
The Best-Approximation Theorem

Thm. Let W be a subspace of $\mathbb{R}^n$, and let $\hat{y}$ be the orthogonal projection of y onto W . Then

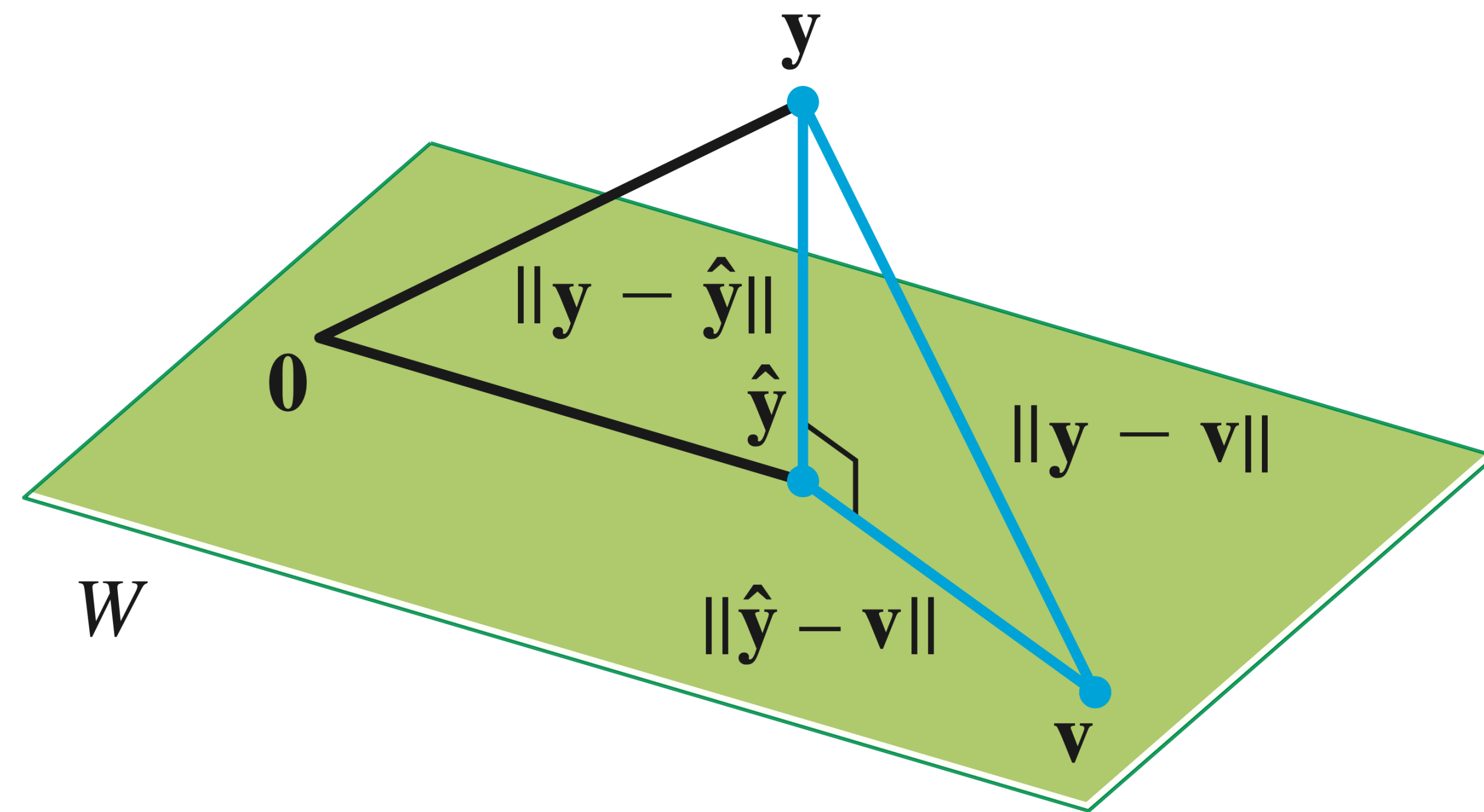
$$\|y - \hat{y}\| \leq \|y - w\|$$

for *any* vector w in W

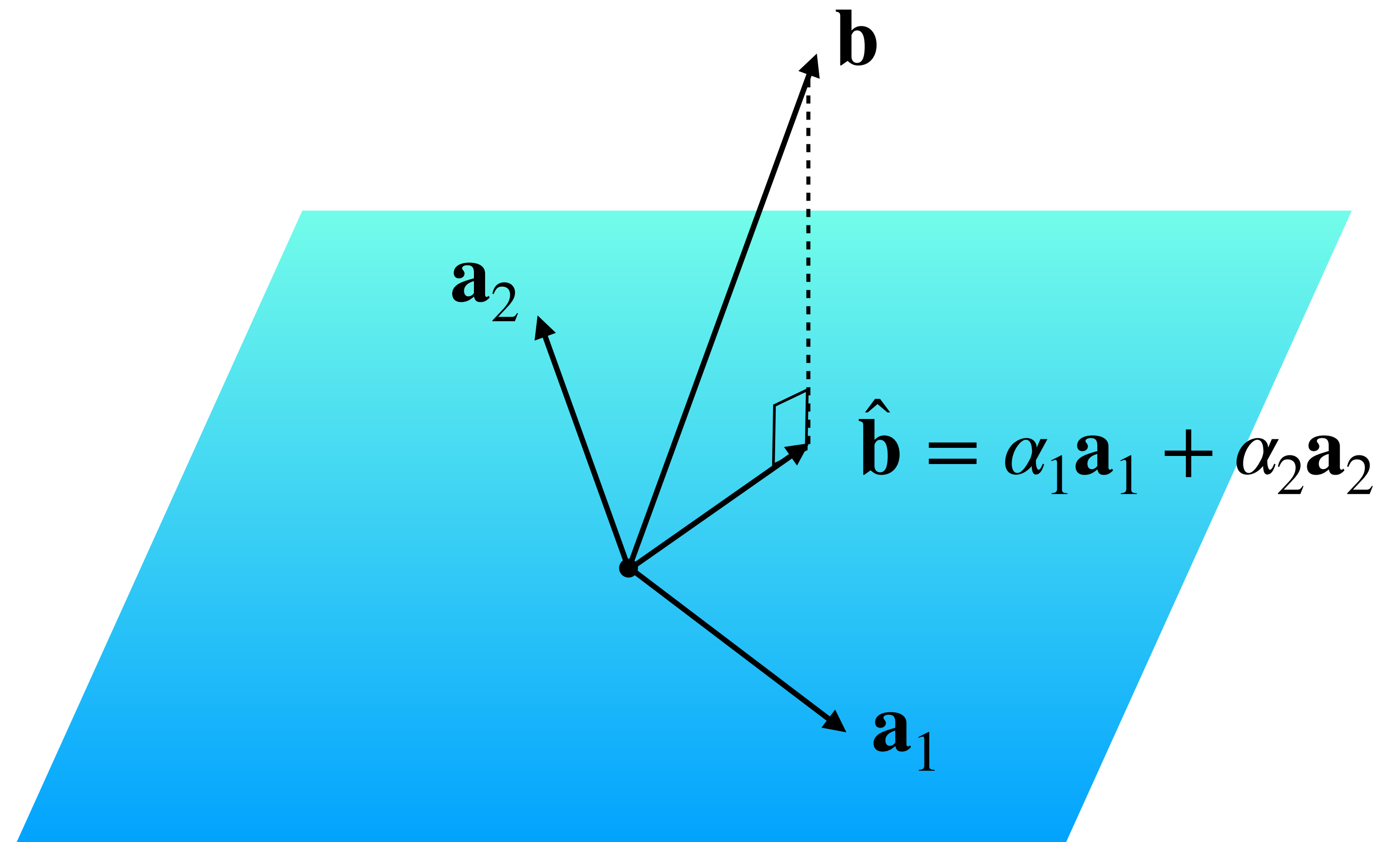
$\hat{y}$ is the closest point in W to y



Proof by Inspection

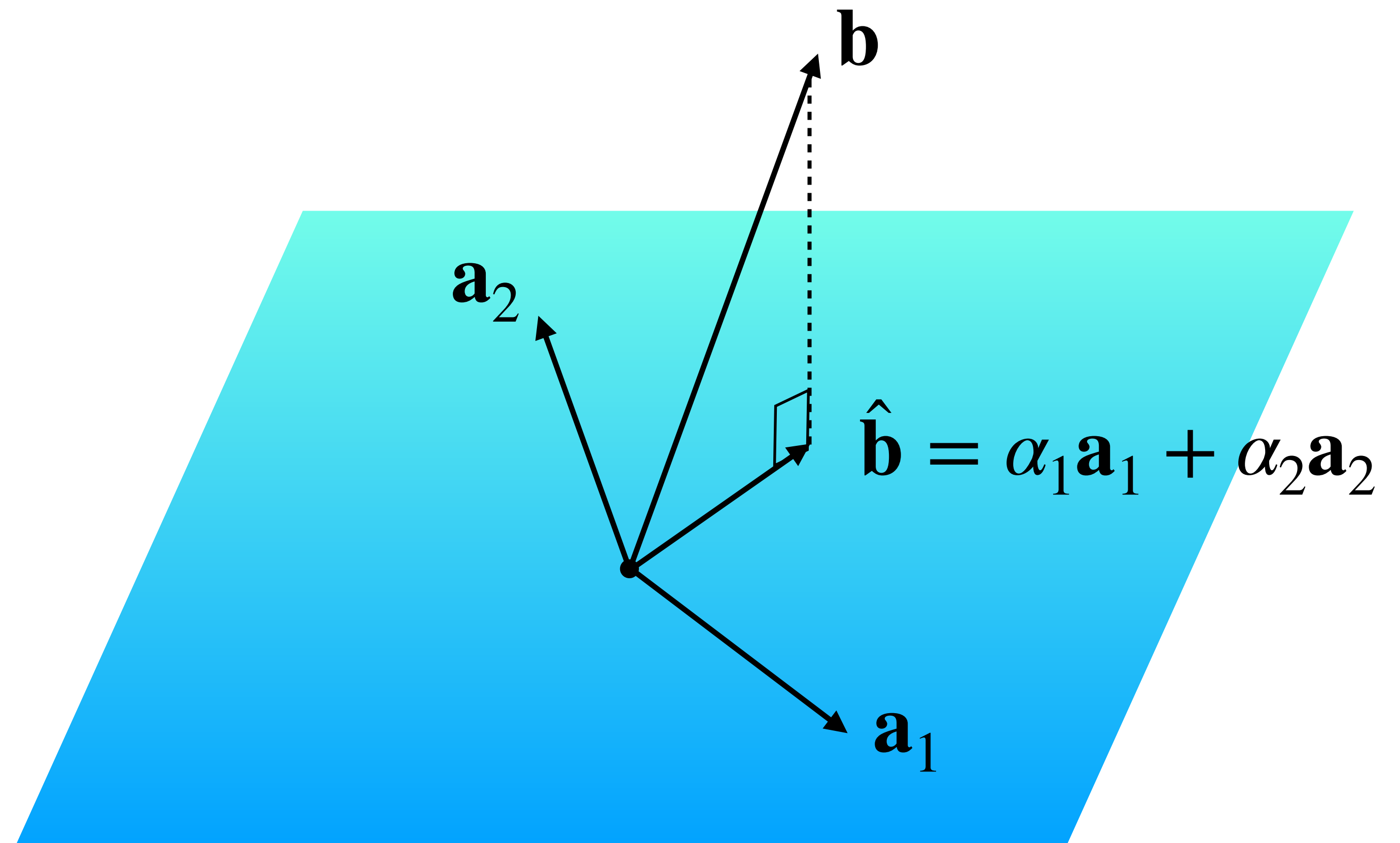


The Point



The Point

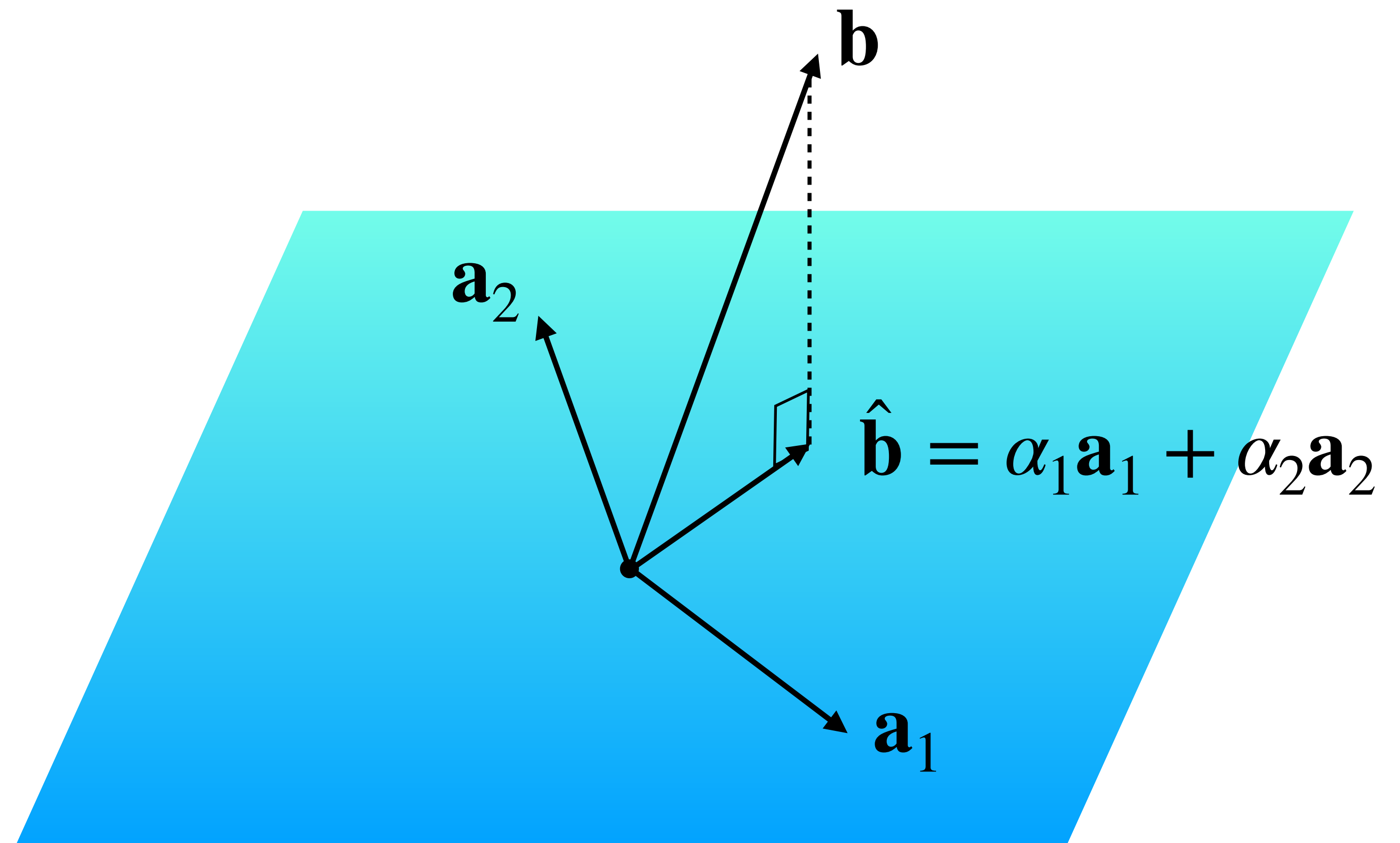
$\hat{\mathbf{b}}$ is in $\text{Col}(A)$ so $A\mathbf{x} = \hat{\mathbf{b}}$ has
a solution



The Point

$\hat{\mathbf{b}}$ is in $\text{Col}(A)$ so $A\mathbf{x} = \hat{\mathbf{b}}$ has
a solution

At this point, we could
call it a day:

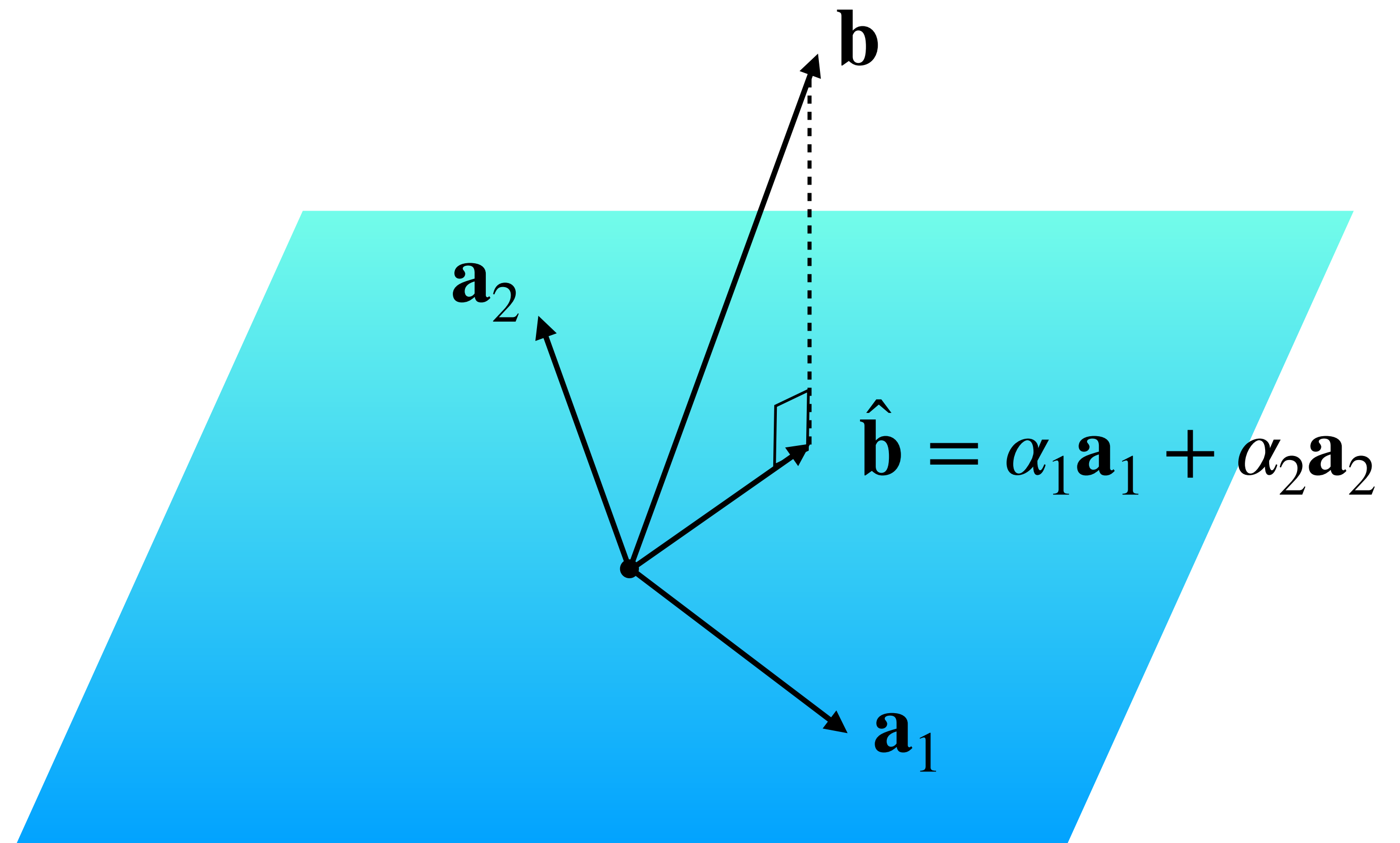


The Point

$\hat{\mathbf{b}}$ is in $\text{Col}(A)$ so $A\mathbf{x} = \hat{\mathbf{b}}$ has
a solution

At this point, we could
call it a day:

Q. Find a least squares
solution to $A\mathbf{x} = \mathbf{b}$



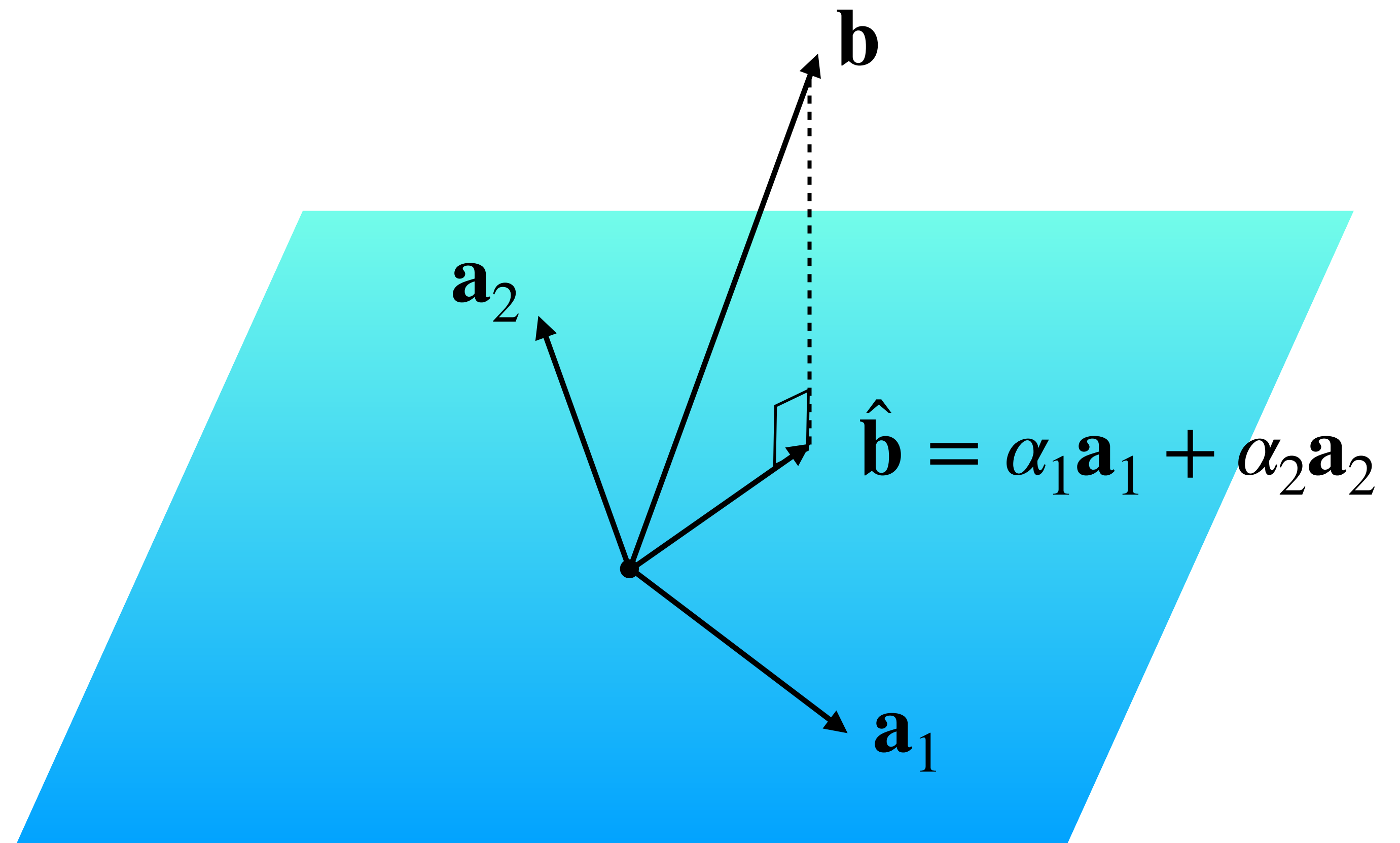
The Point

$\hat{\mathbf{b}}$ is in $\text{Col}(A)$ so $A\mathbf{x} = \hat{\mathbf{b}}$ has
a solution

At this point, we could
call it a day:

Q. Find a least squares
solution to $A\mathbf{x} = \mathbf{b}$

A. Find $\hat{\mathbf{b}}$, then solve
 $A\mathbf{x} = \hat{\mathbf{b}}$

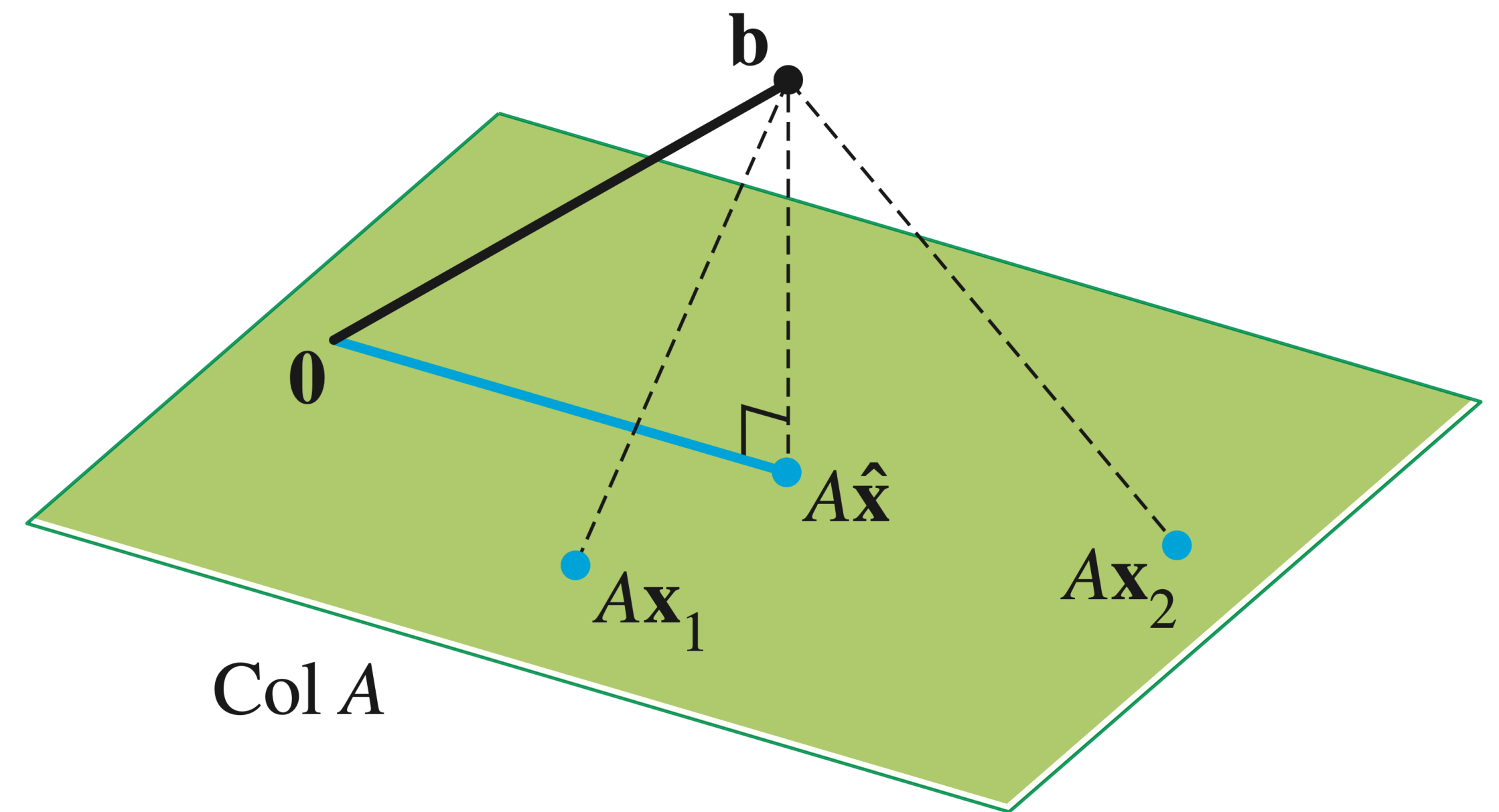


Example

$$\begin{bmatrix} 1 & 2 \\ -1 & 3 \\ 0 & 0 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 4 \\ 1 \\ 4 \end{bmatrix}$$

The Normal Equations

A Couple Observations



Simplify

$$A^T(A\hat{\mathbf{x}} - \mathbf{b})$$

The Normal Equations

The Normal Equations

Thm. The set of least-squares solutions of $A\mathbf{x} = \mathbf{b}$ is the same as the set of solutions to

$$A^T A \mathbf{x} = A^T \mathbf{b}$$

The Normal Equations

Thm. The set of least-squares solutions of $A\mathbf{x} = \mathbf{b}$ is the same as the set of solutions to

$$A^T A \mathbf{x} = A^T \mathbf{b}$$

In particular, this set of solutions is nonempty

The Normal Equations

Thm. The set of least-squares solutions of $A\mathbf{x} = \mathbf{b}$ is the same as the set of solutions to

$$A^T A \mathbf{x} = A^T \mathbf{b}$$

In particular, this set of solutions is nonempty

(We just showed that if $\hat{\mathbf{x}}$ is a least squares solution then $A^T A \hat{\mathbf{x}} = A^T \mathbf{b}$)

Example

$$A = \begin{bmatrix} 4 & 0 \\ 0 & 2 \\ 1 & 1 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 2 \\ 0 \\ 11 \end{bmatrix}$$

Example

$$\begin{bmatrix} 1 & 2 \\ -1 & 3 \\ 0 & 0 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 4 \\ 1 \\ 4 \end{bmatrix}$$

Unique Least Squares Solutions

Question

Is a least squares solution unique?

Answer: No

If $\mathbf{b} \in \text{Col}(A)$ then $\hat{\mathbf{b}} = \mathbf{b}$ and then we're asking if $A\mathbf{x} = \mathbf{b}$ has a unique solution for any choice of A

When is there a unique solution?

The least squares method gives us to find an approximate solution when there is no exact solution

But it doesn't help us choose a solution in the case that there are many

Practically Speaking

numpy.linalg.lstsq

`linalg.lstsq(a, b, rcond='warn')`

[\[source\]](#)

Return the least-squares solution to a linear matrix equation.

Computes the vector x that approximately solves the equation $a @ x = b$. The equation may be under-, well-, or over-determined (i.e., the number of linearly independent rows of a can be less than, equal to, or greater than its number of linearly independent columns). If a is square and of full rank, then x (but for round-off error) is the “exact” solution of the equation. Else, x minimizes the Euclidean 2-norm $||b - ax||$. If there are multiple minimizing solutions, the one with the smallest 2-norm $||x||$ is returned.

Parameters: a : (M, N) *array_like*

“Coefficient” matrix.

b : $\{(M,), (M, K)\}$ *array_like*

Ordinate or “dependent variable” values. If b is two-dimensional, the least-squares solution is calculated for each of the K columns of b .

$rcond$: *float. optional*

Practically Speaking

numpy.linalg.lstsq

`linalg.lstsq(a, b, rcond='warn')`

[\[source\]](#)

Return the least-squares solution to a linear matrix equation.

Computes the vector x that approximately solves the equation $a @ x = b$. The equation may be under-, well-, or over-determined (i.e., the number of linearly independent rows of a can be less than, equal to, or greater than its number of linearly independent columns). If a is square and of full rank, then x (but for round-off error) is the “exact” solution of the equation. Else, x minimizes the Euclidean 2-norm $||b - ax||$. If there are multiple minimizing solutions, the one with the smallest 2-norm $||x||$ is returned.

NumPy chooses the shortest vector

Parameters: a : (M, N) array_like

“Coefficient” matrix.

b : $\{(M,), (M, K)\}$ array_like

Ordinate or “dependent variable” values. If b is two-dimensional, the least-squares solution is calculated for each of the K columns of b .

$rcond$: float. optional

Practically Speaking

numpy.linalg.lstsq

`linalg.lstsq(a, b, rcond='warn')`

[\[source\]](#)

Return the least-squares solution to a linear matrix equation.

Computes the vector x that approximately solves the equation $a @ x = b$. The equation may be under-, well-, or over-determined (i.e., the number of linearly independent rows of a can be less than, equal to, or greater than its number of linearly independent columns). If a is square and of full rank, then x (but for round-off error) is the “exact” solution of the equation. Else, x minimizes the Euclidean 2-norm $||b - ax||$. If there are multiple minimizing solutions, the one with the smallest 2-norm $||x||$ is returned.

NumPy chooses the shortest vector

Parameters: $a : (M, N)$ *array_like*

“Coefficient” matrix.

$b : \{(M,), (M, K)\}$ *array_like*

Ordinate or “dependent variable” values. If b is two-dimensional, the least-squares solution is calculated for each of the K columns of b .

$rcond : float$ *optional*

(why?...))

Unique Least Squares Solutions

Thm. For $A \in \mathbb{R}^{m \times n}$ the following are equivalent:

- » $A\mathbf{x} = \mathbf{b}$ has a *unique* least squares solution for any choice of $\mathbf{b}$
- » The columns of A are *linearly independent*
- » $A^T A$ is *invertible*

Unique Least Squares Solutions

$$\hat{\mathbf{x}} = (A^T A)^{-1} A^T \mathbf{b}$$

If A has linearly independent columns, then its unique least squares solution is the above

Projecting onto a subspace

$$\hat{\mathbf{b}} = A\hat{\mathbf{x}} = A(A^T A)^{-1}A^T \mathbf{b}$$

If the columns of A are linearly independent, then **they form a basis**

Said another way: if $\mathcal{B}$ is a basis, then we can construct a matrix A whose columns are the vectors in $\mathcal{B}$

This means we can find arbitrary projections